Markov Brains for Robotic Swarm Control in Space Exploration

by

Ketan Vasudeva

Supervisor: Gabriele D'Eleuterio
April 2023

B.A.Sc. Thesis



Division of Engineering Science
UNIVERSITY OF TORONTO

Abstract

The recent boom of interest in planetary space exploration has created an increasing need to develop new and robust autonomous robotics solutions to aid in the exploration of other celestial bodies. With most space agencies focusing their sights on the Moon and then Mars, it has become increasingly important to create cheap, safe, and highly redundant autonomous systems to aid in the prospecting of resources. Caves are an interesting resource for prospecting since small caves are not detectable by satellite imagery and they offer natural protection from radiation for potential long-term settlements. Swarm robotics is a particularly interesting domain that can produce inherently redundant autonomous systems based on cheap and simple robots capable of exploration tasks. Markov Brains are a type of representation capable of creating swarm controllers through evolutionary algorithms. This thesis finds that although swarms of Markov Brains can produce cave search and detection behaviours, it may not do so as effectively as other more consistent non-evolutionary algorithms. For the evolutionary algorithm, a correlation is found between the number of generations and fitness, as well as the mutation probability and fitness. Finally, this thesis identifies the lack of a generalized mathematical framework for the analysis and design of Markov Brains as a barrier to their adoption in engineering and more specifically the space industry.

Acknowledgements

The completion of this thesis would not have been possible without the guidance and supervision of Professor Gabriele D'Eleuterio. Thank you for all the time you have invested in exploring this research area along with me.

I would further like to thank the University of Toronto Engineering Science Department for their continued support of the undergraduate thesis project.

Finally, I would like to thank all my friends and family for their support and love throughout this year.

Table of Contents

ABSTRACT **2**

ACKNOWLEDGEMENTS **3**

SYMBOLS, FIGURES, AND TABLES **6**

SYMBOLS 6
FIGURES 6
TABLES 6

CHAPTER 1: INTRODUCTION **7**

1.1. BACKGROUND AND SIGNIFICANCE 8
1.2. OBJECTIVES 8

CHAPTER 2: BACKGROUND **10**

2.1. SWARM ROBOTICS 11
2.1.1. Swarm Behaviours *15*
2.2. EVOLUTIONARY ALGORITHMS 17
2.2.1. Evolutionary Algorithms in Swarm Robotics *19*
2.3. MARKOV BRAINS 21
2.3.1. Gates *24*
2.3.2. Encoding *25*
2.3.3. Evolution *26*
2.3.4. Implementations and History *27*
2.3.5. Analysis *28*

CHAPTER 4: MARTIAN ROBOTIC SPACE EXPLORATION **31**

CHAPTER 5: METHODS **34**

5.1. EXPERIMENT SETUP 35
5.1.1. Cave Maps *35*
5.1.2. Markov Brain Agents *36*
5.1.3. Evolutionary Algorithm *38*
5.2. SIMULATION SETUP 40
5.3. ANALYSIS 42

CHAPTER 6: EXPERIMENT RESULTS **44**

6.1. FITNESS AND HYPERPARAMETERS 45
6.2. SIMULATED BEHAVIOUR 54

CHAPTER 7: DISCUSSION **59**

7.1. DISCUSSION OF ANALYSIS 60
7.1.1. Discussion of Hyperparameter Analysis *60*

7.1.2. Discussion of Cave Detection Swarm Behaviour and Comparisons to the Honeybee Search Algorithm *62*
7.2. EVOLUTIONARY ALGORITHMS AND MARKOV BRAINS 63
CHAPTER 8: CONCLUSION **65**
REFERENCES **68**

Symbols, Figures, and Tables

Symbols

α: fitness
σ: standard deviation
σ^2: variance

Figures

FIGURE 1: ARNOLD ET AL. DEFINITION OF A ROBOTIC SWARM. 12
FIGURE 2: EXAMPLES OF SYSTEMS WHICH ARE NOT SWARM ROBOTIC SYSTEMS 13
FIGURE 3: BRAMBILLA ET AL. SR TAXONOMY 15
FIGURE 4: SLOSS AND GUSTAFSON EA PROCESS EXAMPLE 17
FIGURE 5: NEURAL NETWORK VS MARKOV BRAIN 22
FIGURE 6: RECURRENT NEURAL NETWORK VS MARKOV BRAIN 22
FIGURE 7: COMPARISON OF DIFFERENT NETWORK TYPES 23
FIGURE 8: GENETIC ENCODING SCHEME ILLUSTRATION 25
FIGURE 9: ILLUSTRATION OF GENETICALLY ENCODED MARKOV BRAIN 26
FIGURE 10: ILLUSTRATION OF SMEAREDNESS IN MBS AND NEURAL NETWORKS 29
FIGURE 11: CAVE MAPS FOR TRAINING 36
FIGURE 12: MARKOV BRAIN STRUCTURE 37
FIGURE 13: MBCAVEEXPLORE SOFTWARE ARCHITECTURE 40
FIGURE 14: EFFECTS OF TOTAL EVOLUTION STEPS ON FITNESS 47
FIGURE 15: EFFECTS OF MUTATION PROBABILITY ON FITNESS 48
FIGURE 16: EFFECTS OF THE NUMBER OF HIDDEN NODES ON FITNESS 49
FIGURE 17: EFFECTS OF THE NUMBER OF GATES ON FITNESS 50
FIGURE 18: EFFECTS OF THE NUMBER OF SWARM AGENTS ON FITNESS 51
FIGURE 19: EFFECTS OF THE TOTAL NUMBER OF SIMULATION STEPS ON FITNESS 52
FIGURE 20: MULTIPLE ITERATIONS OF DEFAULT MODEL 53
FIGURE 21: SIMULATION RESULTS FOR MUTATION PROBABILITY 0.03 55
FIGURE 22: SIMULATION RESULTS FOR 750 SIMULATION TIME STEPS 56
FIGURE 23: SIMULATION RESULTS FOR 15 GATES 57

Tables

TABLE 1: COLLECTIVE SWARM ROBOTICS BEHAVIOURS 16
TABLE 2: MARKOV BRAIN GATE TYPES 24
TABLE 3: EXAMPLE IMPLEMENTATIONS OF MARKOV BRAINS 28
TABLE 4: HYPERPARAMETER, FITNESS, AND TRAINING TIME MATRIX 46
TABLE 5: SUMMARY OF EFFECT OF GATE SIZE ON FITNESS 47

Chapter 1: Introduction

1.1. Background and Significance

With plans to go back to the Moon by 2024 and Mars by the late 2030s, the development of new robotic space exploration technologies is becoming increasingly important [1,2]. For Mars, NASA envisions sending materials ahead of time to increase the odds of mission success [2]. Along with these supplies, one could imagine a mission where robotic systems are sent ahead of time, or with the astronauts, to aid in exploration and the establishment of a base on the red planet. A good candidate for such robotic technology is swarm robotics, with potential to produce cheap and highly redundant robotic systems capable of exploration and building tasks [3].

Swarm robotics is a subset of swarm intelligence (the study of collective behaviours), focusing on the control of simple robots in *multiagent* systems. The robots in these systems typically operate in a decentralized manner, with local control algorithms taking inspiration from swarm intelligence. One such type of control involves using evolutionary algorithms, such as with Markov Brains, to manage the behaviour of the robots. Markov Brains use Darwinian evolutionary processes to evolve a robot genome over many generations based on a particular fitness function, with the potential to create dynamic behaviours such as exploration [4,5]. However, Markov Brain research is still limited and contained within a small but active community. As such, the goal of this research is to investigate the creation of exploratory behaviours for swarm robotics using Markov Brains, within the context of robotic space exploration.

1.2. Objectives

The goal of this research is to investigate the use of a Markov to control robotic swarms and develop exploratory behaviours useful in robotic space exploration, particularly for potential missions to Mars. This research will seek to develop, test, and analyse a Markov Brain control technique for this purpose. The objectives can be broken down as follows:

1. Identify useful exploratory behaviours for robotic space exploration, particularly for the exploration of Mars.

2. Develop a Markov Brain to implement **one** of the exploratory behaviours from Objective 1.
3. Simulate the Markov Brain.
4. Analyse the results from Objective 3 with the goal of advancing research in the field.

Chapter 2: Background

This research subject requires drawing on a multidisciplinary field of literature. The goal of this review will be to give an overview of existing literature in the swarm robotics, evolutionary algorithms, Markov Brains, and Martian robotic space exploration research fields. All four of these areas are key to fully understanding the research gap and framing the research within the context of existing literature.

2.1. Swarm Robotics

Swarm robotics (SR) is a combination of the swarm intelligence (SI) and robotics fields [7]. The swarm intelligence field focuses on the study of the self-organized behaviours of social animals, seeking an algorithmic understanding of how complex *multi organism* behaviours are created [8]. Swarm robotics seeks to apply SI concepts to *multi robot* systems, coordinating large numbers of relatively simple robotic agents to produce swarm behaviours [8, 9]. Often swarm robotics takes direct biological inspiration, attempting to replicate the behaviours of a flock of birds, a colony of ants, or a hive of bees [10]. There are many definitions for what comprises a swarm of robots, most of which share similar but slightly varying properties. Spezzano [7] defined the properties of a robotic swarm as:

1. *Autonomous*: Individuals of the swarm-robotic system are autonomous. They can interact with each other and the environment.
2. *Large number*: Swarm robots are in large numbers to cooperate with each other.
3. *Scalable and robust*: New units can easily be added to the system and more units improve the performance of the system. The system is robust to losing some units since there are still units left to perform.
4. *Decentralized coordination*: Swarm robots communicate with each other and with the environment to make decisions and perform actions.
5. *Flexible*: Swarm-robotic systems have the ability to generate modular solutions to various tasks.

This is similar to a simpler definition provided by Brambilla et al. [8]:

1. Robots are autonomous.
2. Robots are situated in the environment and can act to modify it.
3. Robots' sensing and communication capabilities are local.

4. Robots do not have access to centralized control and/or to global knowledge.
5. Robots cooperate to tackle a given task.

This definition is much broader and allows for a much wider interpretation of what consists of a robotic swarm. Finally, a definition proposed by Arnold et al. [10] is displayed in Figure 1. It generally states that swarms should come in groups of 3 or more, should have limited human control, and should be cooperative.

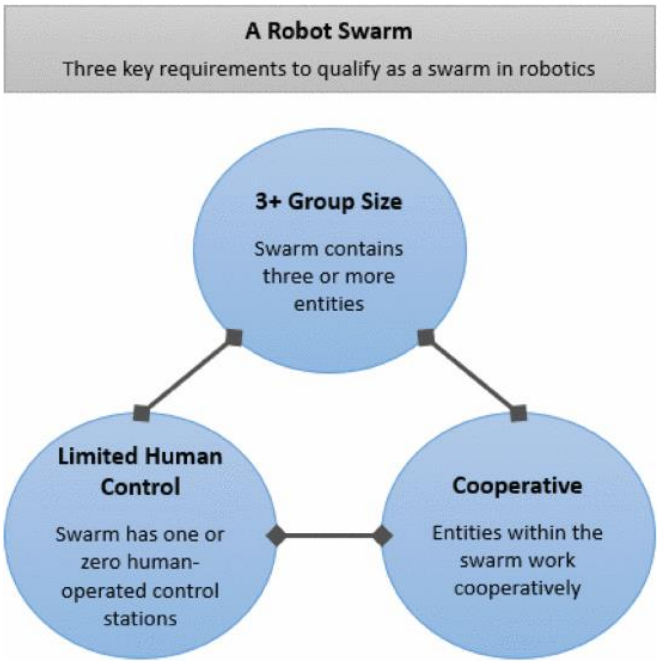


Figure 1: Arnold et al. [10] definition of a robotic swarm.

However, along with these broader definitions, Arnold et al. [10] also note the importance of defining exactly what is not a robotic swarm. For instance, Figure 2 indicates instances where systems are not swarm robotic systems.

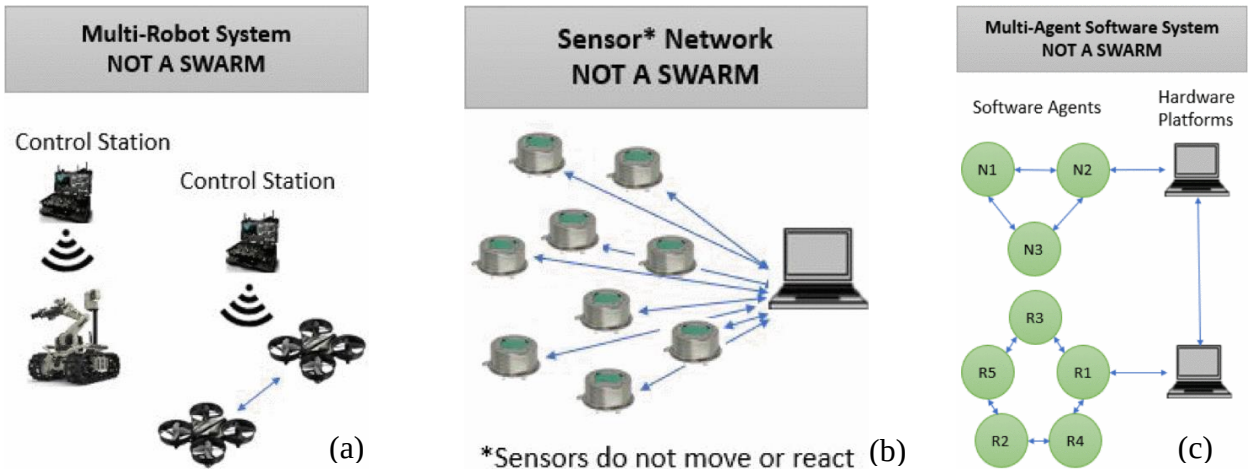


Figure 2: Examples of systems which are not swarm robotic systems [10]

Figure 2a displays how although a system of robots may cooperate to achieve overarching goals, since they work seperately on their tasks, they do not qualify as a swarm robotic system. Figure 2b demonstrates that although a system of sensors may communicate with each other, each unit is not considered a robotic agent in and of itself, disqualifying it as a swarm robotic system. Similar to the sensor network, Figure 2c indicates that a multiagent software system does not qualify as a swarm robotic system since each unit cannot be classified as a physical robot [10]. Note that due to lack of appropriate hardware, most experiments still use simulation in order to evaluate the effectiveness of robotic swarm algorithms. However, these experiments model either physical robotic agents or artificial biological agents in a simulated software world.

Some types of robotic swarms include [9]:

- Unmanned aerial vehicle (UAV) swarms – one of the most popular swarm types due to the increasing capabilities of UAVs and ability to avoid difficult terrain.
- Ground-based robotic swarms – particularly popular for construction or space exploration tasks.
- Water-based robotic swarms

Robotic swarms can be categorized as either heterogeneous or homogeneous. *Homogeneous systems* are composed entirely of identical robots and entities with identical capabilities and/or tasks. *Heterogenous systems* are composed of varying robots and entities with differing capabilities and/or tasks [8,11].

Common potential uses for swarm robotics includes disaster recovery, defense, reconnaissance, inspection, mapping, farming, and space systems [10]. Research in SI and SR has also indirectly led to advances in other fields through biologically inspired algorithms. Examples of this include the popular particle swarm optimization algorithm [12], the artificial bee colony algorithm [13], and the beetle swarm optimization algorithm [14].

Dorigo et al. [11] describes a timeline for swarm robotics, with the first reference being made in 1991 and with popularity rising in academic literature around 2003. Initial studies were aimed at studying *stigmergy*, which is a form of indirect communication between agents where the agents leave traces in the environment that influence other future agents. In the early 2000s, several studies sought to design swarm behaviours, such as foraging [15], stick pulling [16], clustering [17], and sorting [18]. These studies coincide with the timing of one of the first internationally funded projects to investigate swarm robotic cooperation by the European Commission, called Swarm-bots, from 2001-2005. This study explored concepts of robotic swarm self-assembly and cooperation to solve a complex task. These ideas were extended with the 2006-2010 Swarmanoid project, extending the concepts to heterogenous swarms composed of flying, climbing, and ground-based robots [11].

2.1.1. Swarm Behaviours

Brambilla et al. [8] describe a taxonomy to classify methods and collective behaviours, as shown in Figure 3.

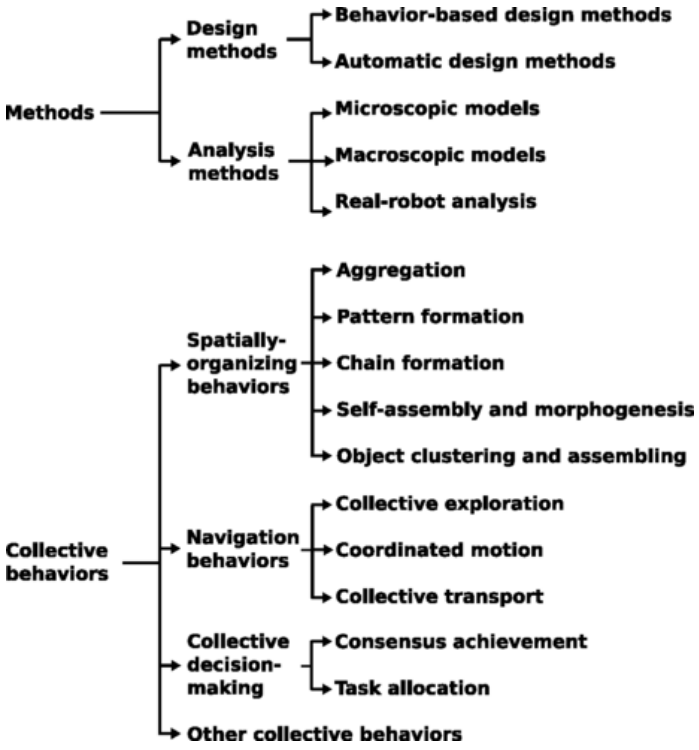


Figure 3: Brambilla et al. SR Taxonomy [8]

Following this taxonomy, there are two main types of method for swarm robotic behaviour design. The first kind is *behaviour-based design*. In this case, the focus is on the iterative improvement of individual robot behaviour based on inspiration from social animals. These methods include [9]:

- Probabilistic finite-state machines (PFSMs)
- Virtual physics-based design – rooted in the idea of an artificial potential field
- Protoswarm – script writing at the collective level
- Property-based design

The second type of method for swarm robotic behaviour design are the *automatic design methods*. These methods do not require the explicit intervention of the developer and instead learn behaviours through other mechanisms. The two main branches of these methods are [9]:

- Reinforcement learning
- Evolutionary robotics

This research focuses on the latter of these two methods, with Markov Brain methods being a subset of evolutionary robotics.

Finally, the behaviours listed in Figure 3 are further explained below in Table 1.

Table 1: Collective Swarm Robotics Behaviours

Swarm Robotic Behaviour	Explanation	Design Methods
Aggregation	Robots group together at a designated location.	PFSMs [19] and artificial evolution [20]
Pattern Formation	Robots organize/deploy themselves into a pattern.	Virtual physics-based design [21,22]
Chain Formation	This behaviour is inspired by ant chains. Robots connect two points by connecting their bodies and forming a chain between the points.	PFSMs [23], virtual physics-based design [24], and artificial evolution [25].
Self-assembly and Morphogenesis	This category of behaviour allows robots to physically connect to one another, or change their structure, in order to perform various tasks.	Decomposed into two issues [26]: 1. Morphogenesis: PFSMs 2. Structure control: artificial evolution and PFSMs
Object Clustering and Assembling	Robots move objects into clusters (physically close) or assemblies (physically connected) configurations.	PFSMs [27]
Collective Exploration	Robots work together to explore a new area. This includes maximizing the area covered (coverage) and guiding robots through the environment (swarm-guided navigation).	Two behaviours [28] [29] [30]: 1. Coverage: virtual physics-based design 2. Swarm-guided navigation: PFSMs
Coordinated Motion (flocking)	Robots move together in a formation/pattern.	Virtual physics-based design [31] and artificial evolution [32]
Collective Transport	Robots work together to transport an object. Usually, the object is too large or massive to be individually transported.	PFSMs [33] and artificial evolution [34].
Consensus Achievement	Allows robots to make decisions among alternatives.	Two different approaches: 1. Direct communication [35] 2. Indirect communication [36]
Task Allocation	Robots organize themselves to distribute tasks.	PFSMs [37]

2.2. Evolutionary Algorithms

Evolutionary algorithms (EAs) are a set of heuristic-based algorithms that leverage the concept of Darwinian evolution to evolve artificial agents/representations [38]. Evolutionary algorithms can be broken down into four main steps:

1. **Initialization:** the first set of candidate solutions are produced, often through random generation.
2. **Selection:** candidate solutions are evaluated based on a fitness function. A fitness function outputs a value indicating the performance of the candidate solutions.
3. **Genetic Operators:** the genetic operators are inspired from natural selection and are used to create new candidate solutions. Typically, there are two main genetic operators:
    a. **Mutation:** parts of a candidate solution are randomly changed to create a new candidate solution.
    b. **Crossover:** two candidate solutions are mixed to create a new candidate solution.
4. **Termination:** the stopping condition for the EA, usually based on runtime or performance.

Sloss and Gustafson [38] undertook a thorough literature review in 2019 describing many of the concepts related to EAs. They defined what a typical EA process might look like (shown in Figure 4) as well as much of the terminology.

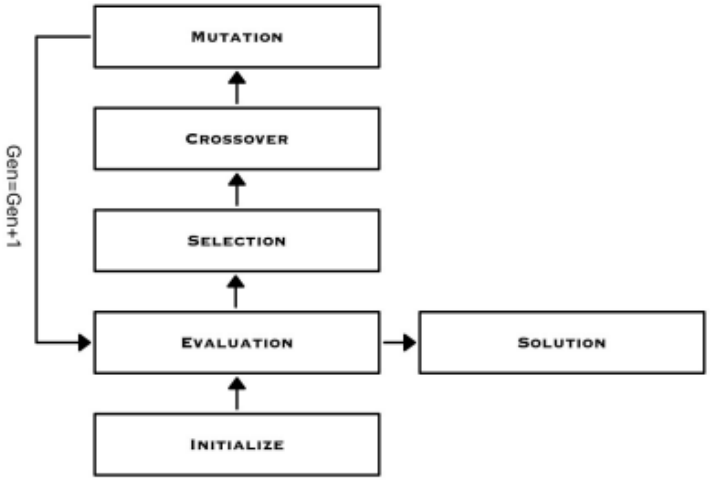


Figure 4: Sloss and Gustafson [38] EA Process Example

Sloss and Gustafson [38] also further explain some of the important concepts for evolutionary algorithms. At its core, an EA works by generating a population of solution candidates or entities, called phenotypes. These entities can be represented in various ways, such as traditional strings or more complex programs. The genotype is what the EA manipulates and varies, and it can take many forms, including strings, tree structures, and graphs [38].

The grammar defines the rules of the EA, including the boundaries for the representation and the valid operations on the genotype. This ensures that the resulting solutions are valid and meet the requirements of the problem [38].

Fitness is a critical concept in evolutionary algorithms as it measures how close a result is to a desired goal. A fitness function computes a fitness value indicating the performance of a candidate solution. These values are then used to determine selection candidates for the next generation [38].

Constraints represent the physical goals of the system, indicating real-world limitations that must be considered when generating solutions. For example, constraints may include code size, energy consumption, or execution time. Logical goals, on the other hand, are the desired behavior itself [38].

Selection is the process of choosing individuals in the current population to be parents for the next generation. A diversity of candidates is required to create a healthy population. Various selection strategies exist, such as tournament-style selection based on fitness values [38].

In niching, individual entities survive generations in distinct areas of the search or solution space. This can be useful when trying to optimize a multi-objective problem, where solutions may exist in different parts of the solution space. In contrast, crowding is a strategy for replacing an individual entity with similar featured individuals [38].

Finally, elitism is a strategy where the strongest entities remain in the population. The parents of the previous generation become the children of the new generation. This ensures that the best solutions are preserved, which can lead to faster convergence [38].

Finally, Sloss and Gustafson [38] also investigate many traditional examples of evolutionary algorithms. One popular EA is Evolutionary Strategies (ES), which is a subset of EAs that uses a

strategy parameter instead of a fixed mutation step size [40]. Another well-known EA is Genetic Algorithms (GA), which employs techniques such as selection, crossover, and mutation to explore the search space of solutions [41]. Genetic Programming (GP) is another type of EA that evolves computer programs instead of fixed-length strings [42]. Genetic Improvement (GI) is a specific form of GP that focuses on improving existing code rather than generating new programs [43]. Grammatical Evolution (GE) is a type of GP that employs context-free grammars to generate variable-length strings representing computer programs [44]. Linear Genetic Programming (LGP) is another form of GP that represents programs as linear sequences of instructions [45]. Finally, Cartesian Genetic Programming (CGP) is a form of GP that represents programs as directed graphs, allowing for efficient use of computer memory [46].

In addition to traditional evolutionary algorithms, Sloss and Gustafson [38] highlight some unique methods based on EA concepts but with more complex and flexible representations. One such method is Auto-constructive Evolution, which evolves both the solution and the representation of the solution [47]. Another approach is Neuroevolution or Deep Neuroevolution, which uses EAs to train artificial neural networks to solve complex problems [48]. Self-replicating Neural Networks is another EA-based approach that uses self-replicating programs to evolve neural networks [49]. Finally, Markov Brains is an approach that evolves the internal computational gates and connections of a flexible network [4]. Markov Brains are further explained in Section 3 of this chapter.

2.2.1. Evolutionary Algorithms in Swarm Robotics

Evolutionary Algorithms (EAs) have been the subject of extensive research in the field of swarm robotics. Türkler et al. [50] have provided a comprehensive overview of swarm algorithms that have been developed and tested for use in robotic swarms. Among these algorithms, Particle Swarm Optimization (PSO) stands out for its ability to mimic the social behavior of birds flocking or fish schooling. By simulating the way that these creatures move and interact with each other, PSO has been shown to be highly effective in finding optimal solutions to a variety of problems.

Ant Colony Optimization (ACO) is another algorithm that has been highly successful in the field of swarm robotics. Based on the foraging behavior of ant colonies, ACO has been used to solve problems related to path planning, coverage optimization, and task allocation. The algorithm works by simulating the way that ants deposit pheromones to mark the paths they travel. By following these pheromone trails, other ants can find the shortest path to food sources. In swarm robotics, ACO can be used to optimize the way that robots move through an environment, ensuring that they reach their targets as quickly and efficiently as possible [50].

Artificial Bee Colony Optimization (ABC) is another algorithm that is widely used in swarm robotics. Inspired by the food foraging behavior of honeybees, ABC has been applied to a wide range of optimization problems. The algorithm works by simulating the way that bees explore an environment in search of food sources. By dividing the search space into smaller regions and assigning bees to explore each region, ABC can quickly converge on an optimal solution. In swarm robotics, ABC can be used to optimize tasks such as task allocation, path planning, and coverage optimization [50].

Finally, Fish School Search Optimization is an algorithm that takes inspiration from the social behavior of fish. By simulating the way that fish interact with each other, this algorithm has been shown to be highly effective in solving a variety of optimization problems. In swarm robotics, Fish School Search Optimization can be used to optimize tasks such as path planning, coverage optimization, and task allocation, ensuring that robots are able to work together effectively to complete complex tasks [50].

2.3. Markov Brains

Markov Brains (MBs) were identified as a good candidate for swarm control owing to the subsumption architecture which directly interfaces with the environment through sensors and actuators. There are also a few examples of swarm control using MBs; however, the breadth and body of the literature on the subject is still relatively small and leaves much room for further research, particularly in aerospace applications.

Hintze et al. [4] describe Markov Brains as a class of evolvable artificial neural networks, differing from traditional ANNs in many aspects. They describe the key difference being that instead of a layered architecture with each node performing the same function, MBs are networks composed of individual computational components. These computational components, their function, their connections to each other as well as sensors and motors, are all evolved via EA.

Hintze, Schossau, and Bohm [5] also describe MBs as a computational substrate comprised of three main elements. These elements are nodes, gates, and wires. *Nodes* are input, output, or hidden values. *Gates* are logical units executing computations between nodes. *Wires* simply connect nodes and gates. One MB cycle executes all the gates in parallel, except any linear combinations of gates. The number of nodes remains constant, however the gates and their connections (wires) are evolved by EA.

One last definition of MBs is proposed by Sheneman and Hintze [51] describing them as evolvable networks of probabilistic and deterministic logic gates. They introduce the concept of the *feedback gate*, which provides unique internal memory and feedback capabilities to MBs. Feedback gates and other gates are further explained in section 3.1. MBs use internal feedback and hidden states in order to emulate internal memory (analogous to working memory or short-term memory), whereby an agent essentially learns and stores information about the environment.

In summary, MBs can be described as evolvable networks comprised of nodes, gates (computational components), and wires. MBs abide by the Markov property [52] and execute gate operations in parallel, save for linear combinations of gates. The gates in an MB are very flexible, and can be composed of deterministic, probabilistic, or other logic. The primary rule for these gates is that they must take as input and output Boolean

(discrete) logic values. MBs can be compared with various forms of neural network, as displayed in Figures 5 and 6. We see that although the input, output, and hidden node structure is very similar to neural networks, the key differences lie in the internal components and connections. The internal components also typically only have a single layer, as opposed to neural networks which can have many hidden layers.

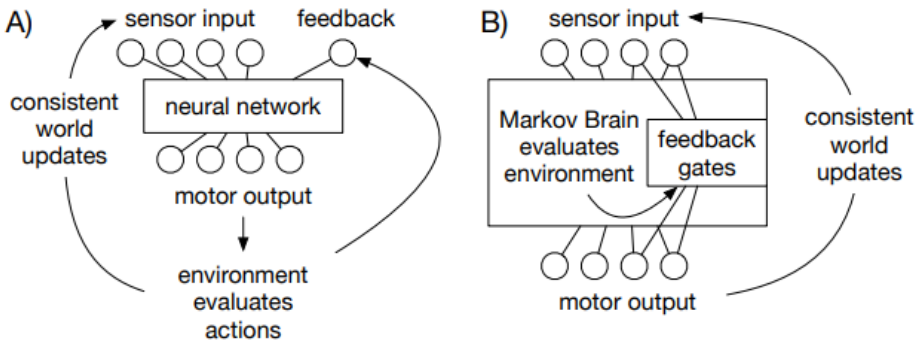


Figure 5: Neural Network vs Markov Brain [51]

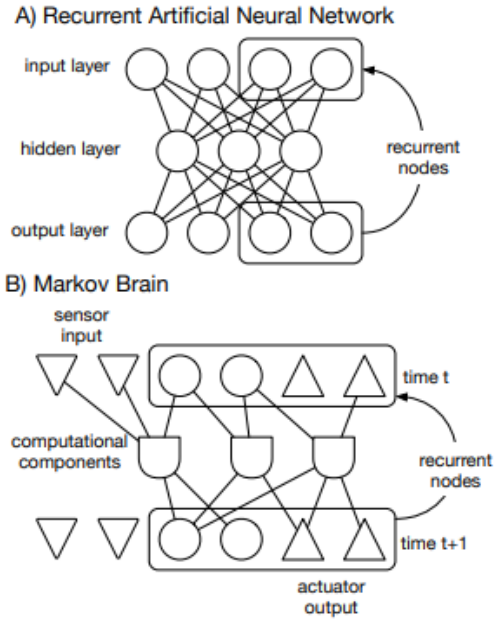


Figure 6: Recurrent Neural Network vs Markov Brain [4]

Another comparison is made by Bohm et al. [53], comparing MBs with both Cartesian Genetic Program (CGP) brains and recurrent neural networks (RNNs), as shown in Figure 7. Cartesian Genetic Programming (CGP) is a type of Genetic Programming (GP) that uses directed graphs as its genotype, where nodes represent functions or inputs, and edges represent connections between them. Recurrent Neural Networks (RNNs) are a

type of neural network that use feedback connections, allowing the network to process sequential data and maintain an internal memory of the previous inputs.

A key difference with the CGP brain is the generalizability of the computational components in the MB. CGP brains have a specific set of CGP gates that can be used. Meanwhile, MB gates can be made as desired as long as they follow the Boolean IO rules. Another key difference when compared to RNNs and CGP brains is the connectivity of the MB. While RNNs are densely connected, MBs are sparsely connected, with no direct connection between the gates themselves (other than linear combinations). This also differs from CGP brains since these can have connections between gates (as shown in Figure 7). This means that computational components read or write directly to the nodes to which they are connected. This sparseness allows MBs to operate each series of computational components in parallel, allowing for parallel updates between timesteps.

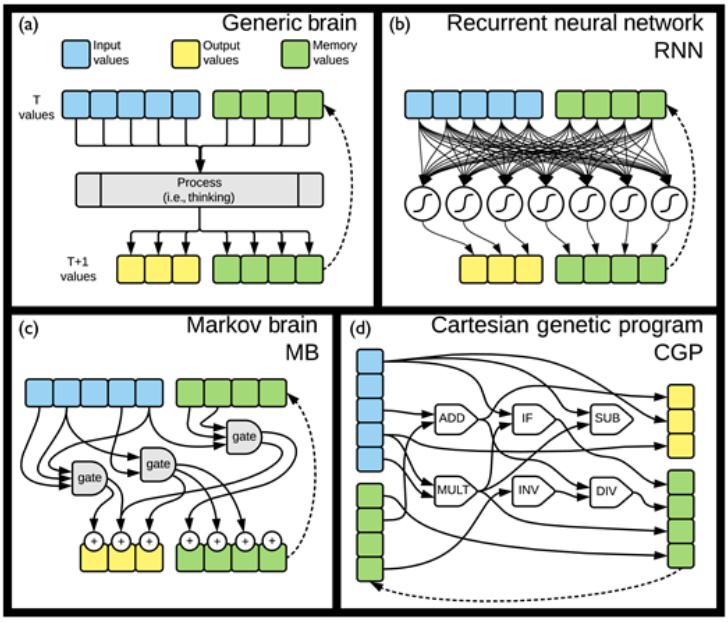


Figure 7: Comparison of Different Network Types [53]

Another key property of MBs, which makes them ideal for robotics implementations, is their ability to interface directly with sensors and actuators. Marstaller et al. [54] refer to this as an example of a subsumption architecture, where autonomous behavior-producing components of a cognitive system directly interface with the world and with each other, rather than with a central symbol processor which deals with explicit representations of

the environment. In other words, MBs are a single-layer system, with the system components directly interfacing with the environment and without the need for additional layers of logic. This makes MBs particularly interesting for swarm applications where large numbers of relatively simple robots/agents can be used to perform complex tasks. Example implementations of MBs are presented in detail in section 3.4.

2.3.1. Gates

A key component to MBs are the computational components or gates which provide the internal logic to the network. Many different types of gates have been created, as presented by Hintze et al. [4] and summarized in table 3 below.

Table 2: Markov Brain Gate Types

Gate Type	Function
Deterministic	Classical logical operations such as AND, NOT, OR, XOR, NAND, etc.
Probabilistic	Similar to deterministic gates, but instead inputs are mapped to outputs via probability tables. Deterministic gates are a special case of probabilistic gates where all probabilities are 0 or 1.
Artificial Neural Network (ANN)	A single layer artificial neural network with a flexible number of inputs and outputs.
Threshold	Discretizes inputs into 0 and 1 and accumulates them, checking if a threshold is reached.
Timer	Does not need inputs and fires 1 on all outputs after a predefined number of brain updates has passed.
Feedback	Similar to probabilistic gates, but with two extra inputs for positive and negative feedback. The feedback changes probability table entries depending on type of feedback. This mechanism is further explained below.
Ternary Logic	Discretizes inputs uniquely, with values greater than 1 being 1, values below -1 being -1, and all values in between being 0. This is a unique example since it does not entirely follow the Boolean IO rules since there are 3 possible results.

Deterministic gates are the easiest and most implemented type of gates. They consist of traditional logic gates, such as AND, OR, and XOR and take binary input to evaluate the binary truth tables. Probabilistic gates are exactly like deterministic gates (in terms of operations), except that they include output probabilities in at the output stage. This

induces stochasticity into the system, allowing the MB to create dynamic behaviours. Probabilities are typically randomly initialized while enforcing the gate type. This means that for an AND gate, the probability of outputting a 1 will be highest when all inputs are 1. This probability is evaluated by taking the inputs, checking what the corresponding output probability is, drawing a random number between 0 and 1, and outputting a 1 if the number is below the output probability for that set of inputs [55].

Feedback gates are a unique type of gate that have proven to increase the performance of MB models [51]. It is usually achieved by relating positive or negative feedback (from the two extra inputs) to changes in the probability table values. In practice, this is implemented by applying changes that increase or decrease probabilities through a state buffer. State buffers store IO pairs over multiple brain updates. This is necessary since outputs from a gate may not result in immediate feedback within one brain update. Depending on how the output impacted the environment, it may take many updates for the effect of the output to impact the system. Therefore, positive and negative feedback is applied to the state buffer. Both the feedback connections and the size of the state buffer are evolvable properties [4]. It is worth noting that feedback connections do not connect to other gates, but rather to input, output, or hidden nodes.

2.3.2. Encoding

Markov Brains can either be genetically or directly encoded in order to be evolved by evolutionary algorithm [4]. Direct encoding involves evolving traditional *String* representations of the MB, while genetic encoding involves evolving a genome representation. A useful property of MBs is that they can be nicely represented by genetic encoding. Hintze et al. [4] provide an example illustration of how one might genetically encode an MB, shown in Figure 8.



Figure 8: Genetic Encoding Scheme Illustration [4]

Genetic encoding involves representing the MB gates, nodes, and wires as strings of numbers. This includes specifying the inputs, outputs and start codons. Start codons are a

unique encoding value that indicates the beginning of a MB element. This value must be unique and carefully selected to avoid confusion when reading the genome. The end of an element is denoted by the next start codon. Hintze, Schossau and Bohm [5] depict a more detailed example of a genetically encoded MB in Figure 9.

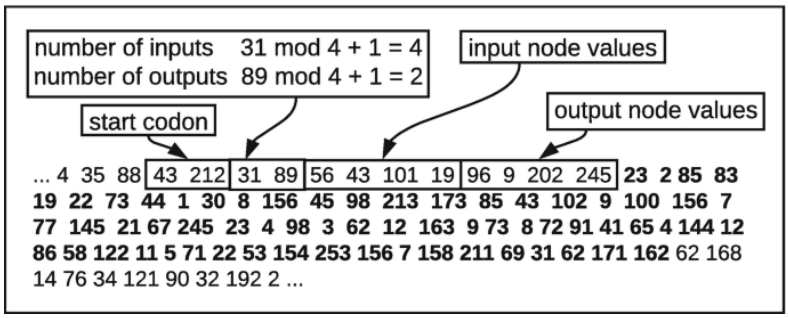


Figure 9: Illustration of Genetically Encoded Markov Brain [5]

Note that specific encoding schemes to represent specific gates, nodes, inputs, outputs, etc. are unique and are determined by the developer of the MB.

2.3.3. Evolution

The evolution of MBs follow the same process as almost all evolutionary algorithms. A typical pipeline for EA evolution was depicted in Figure 4. The evolvable properties of an MB can be summarized as:

- Type and number of computational components, noting that the type must be selected from pool of valid gates (within allowable grammar)
- All wires/connections
- State buffer size and state buffer update rule (magnitude of feedback)

The non evolvable properties of MBs are:

- Number of input, output, and hidden nodes – hidden nodes in MBs are input nodes that are not connected to external input. Instead, these are written directly by some combination of output nodes.

Schossau et al. [56] also investigated the usage of neuro-correlates to boost the performance of evolutionary algorithms. Neuro-correlates are measures from cognitive science that attempt to measure how well a brain/network is working, independently of fitness. The goal was to incorporate neuro-correlates into the fitness and selection process to increase performance. Results found that while certain neuro-correlates, such as R (see

section 3.5), increased the performance, they did not all succeed in doing so. However, this does indicate that there is potential for neuro-correlates to augment the evolutionary process. Neuro-correlates are further explained in section 3.5 as a method for analysis of MBs.

2.3.4. Implementations and History

Markov Brains are a relatively new concept with a small but active research community. They are inspired by Markov models, with most research leading back to the 2009 textbook *Probabilistic and Graphical Models* by Daphne Koller and Friedman Nir [57] as a foundation for initial concepts. Initial research was largely biologically inspired, with MBs being used as a tool to understand biological evolution. This gave rise to some of the first MBs in the form as it can be found today in 2011 with Edlund et al. [55] implementing a maze navigation algorithm using MBs.

Olson et al. [58] furthered this research by producing an experiment simulating predator-prey behaviour in 2013 by coevolving 2 different sets of MBs. This was also the first example of MBs being used to implement swarm behaviour. The same year also marked the appearance of the Modular Agent-Based Evolver (MABE) tool, seeking to provide a standardized research platform for evolutionary brain techniques, such as MBs [6]. The platform is well documented and maintained, making it a potentially invaluable resource for future research.

Major literature reviews detailing the workings and examples of MBs began appearing from 2017 onwards. Major reviews include Sheneman et al. (2017) [51], Hintze et al. (2017) [4], and Hintze, Schossau and Bohm (2019) [5]. By this point, many simulated implementations of MBs had been researched, as summarized by Table 3.

Table 3: Example Implementations of Markov Brains

Year	Experiment	Reference Literature
2011	Edlund et al. implement an MB on an animat capable of maze navigation.	[55]
2013	Marstaller et al. implement an MB to solve a block catching and categorization task.	[54]
2013	Olson et al. simulate predator-prey behaviour using coevolved MBs. This is the first implementation of MBs for swarm behaviour.	[58]
2015	Kvam et al. implemented MBs to solve a binary decision-making task. They examined the effects of environment setup and task difficult on brain size and complexity.	[59]
2018	Fischer et al. implement MBs to control a swarm of animats undertaking a task where they have to cross between two rooms through a small hallway without colliding. The paper investigated the effects of swarm size on the success of the swarms. The MBs from this implementation were able to produce swarm behaviour through strategies to avoid collisions.	[60]
2018	Tehrani-Saleh et al. implemented MBs for motion detection and produced behaviour similar to the Reichardt detector.	[52]
2019	This paper by Hintze et al. tackles an assortment of tasks using MBs. These include balancing an inverted pendulum, value judgement (decision making), block catching, associative memory, and maze solving.	[5]

Finally, Hintze and Schossau [61] began to address the question of hardware implementations in 2022, attempting to design an FPGA which would compute MB brain updates in one clock cycle. This research is still ongoing and seeks to overcome the hardware limitations that pose a barrier to practical physical implementation of MBs.

2.3.5. Analysis

Along with the implementation of MBs, much of the existing literature focuses on analysis of MBs in order to assess their performance and further understand their internal workings. Unfortunately, like neural networks, MBs are considered to be epistemologically opaque [62]. This means that it is difficult to explain the evolved logic of MBs using traditional common narratives like causal logic. Instead, researchers have focuses on creating statistical measures to investigate information distribution within MBs.

One of these measures is the integrated information statistic Φ, which measures how much information is lost if a subsystem is partitioned in any way. Albantakis [63] argues that removing sensors and actuators reveals an integrated system inside the MB.

Schossau et al. [56] investigate MBs from the perspective of neuro-correlates, which are statistical measures of the effectiveness of a particular representation. Some proposed neuro-correlates include the Minimum Description Length (MDL), topological complexity, connectivity and sparseness measured by the Gamma Index (GI), the R statistic (a measure of how much information is stored in a brain's internal states), Information Integration using Φ, and Predictive Information (PI). Neuro-correlate R is one of the most common statistics and quantifies the amount of information an agent has about its environment that is not currently provided by its sensors.

Kirkpatrick et al. [64] introduce the concept of smearedness as an analysis method for MBs. Smearedness is the amount of representation/information that each memory state in the brain has about each individual concept in the environment. They argue that MBs suffer much less from smearedness than neural networks (Figure 10).

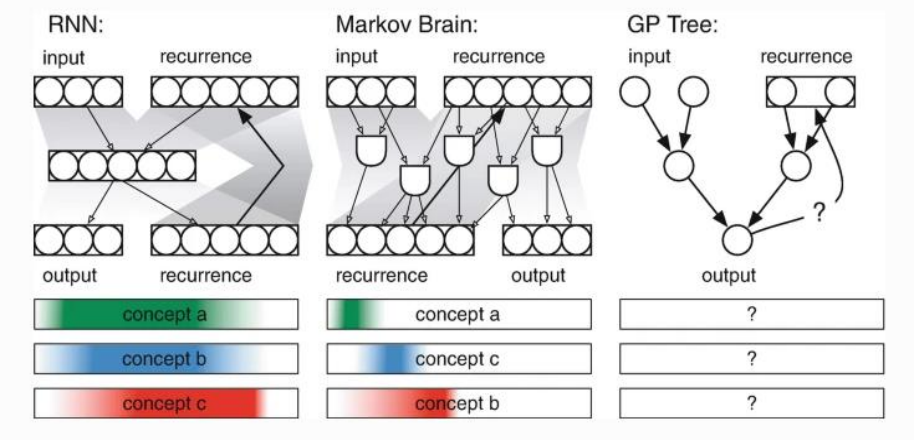


Figure 10: Illustration of Smearedness in MBs and Neural Networks [64]

Edlund et al. [55] also discuss line-of-descent analysis and knockout analysis as common analysis methods. Line-of-descent analysis is used to investigate the evolution of a particular trait or behavior across multiple generations by keeping track of the parents in each generation. Knockout analysis involves temporarily removing a particular set of nodes or gates from the MB to observe how this affects the overall behavior of the

system. This helps researchers understand how different nodes and gates influence the behaviour and success of the system.

Most of the research into these statistical measures has been largely theoretical and often lacks specific details on how they might be implemented. This is a major barrier to the use of most of these analysis methods. This lack of detail means that most of these analysis methods are not easy to intuitively understand and implement and thus will not be heavily used in this thesis.

Chapter 4: Martian Robotic Space Exploration

The final consideration for this research is to consider how these technologies can be useful within the greater context of Martian robotic space exploration. Almost all concepts for future Mars exploration involve some level of robotic exploration as a precursor to human arrival. It has been proposed that robotic outposts could prepare for human arrival on Mars by deploying power stations, assembling habitats, and prospecting for resources. NASA has typically taken a "follow the water" strategy to long-term human exploration of Mars, requiring detailed prospecting and characterization of resources on Mars [65, 66]. Robots will also be needed to aid with in-situ resource utilization for long-term term missions on the planet.

In general, there have been 6 successful Mars rover missions to date: Spirit, Sojourner, Opportunity, Curiosity, Perseverance, and Tianwen 1. All these rovers have been traditional 6-wheeled rovers. Petrovsky et al. [67] argue that these rovers have not kept up with robotic advancements on Earth and require an increase in mobility, range, and lifetime. They further argue that modular or swarm robotic systems are more effective and robust over singular solutions for tasks such as target searching or environment explorations. Perseverance and the Ingenuity helicopter have been helping pioneer the idea of modular systems on Mars [67]. This has led to UAV concepts for Martian exploration, such as St-Onge et al.'s [68] guided robotic swarm. St-Onge et al. [68] indicate that UAVs would have the advantage of ignoring complex terrain, allowing for greater area coverage.

Kisdi et al. [69] also propose and test a robotic swarm using the honeybee search algorithm for the purpose of detecting cave entrances on the surface of Mars. The honeybee search algorithm is a metaheuristic optimization technique that is inspired by the foraging behavior of honeybees. The algorithm mimics the process of scouting for food sources and communicating their locations to other bees through a dance language, while also incorporating a process of abandoning unsuccessful food sources and exploiting promising ones.

Caves are particularly interesting for prospecting on Mars since they provide natural shielding from radiation and milder/stable temperatures. Although orbital imagery can find large cave entrances, smaller usable caves need to be explored closer to the surface. Caves also have detectable temperature gradients when compared with the surface

temperature. They are warmer during the night and cooler during the day. This provides a unique property by which swarm agents might localize possible cave sites. The experiment by Kisdi et al. [69] is used as inspiration for our Markov Brain experiment since it provides an example of a useful swarm behaviour for planetary exploration.

In general, robotic swarms are highly redundant systems composed of many cheaper and smaller robots. This makes them ideal for prospecting or exploration tasks, particularly in an environment where human exploration is not yet possible, such as Mars. While some robotic swarm concepts for space exploration have been simulated, none have attempted to use Markov Brains for exploration. This makes for a novel research area for further investigation.

Chapter 5: Methods

5.1. Experiment Setup

The goal of this experiment is to design a Markov Brain capable of exploratory behaviour for use in robotic space exploration. The inspiration is taken from the aforementioned experiment by Kisdi et al. [69] which used the honeybee search algorithm to control a swarm of agents as they find caves on Mars. Implementing the experiment under similar conditions will enable the comparison of the success of the Markov Brain agents when compared to the honeybee search agents. While the general task objective is to detect caves, the ideal behaviour would be to produce agents capable of randomly searching (ex. Random walk) the map for caves while avoiding collisions.

5.1.1. Cave Maps

The experimental setup consists of a 2D 1 km^2 area modeled by a 500 x 500 pixel temperature map. There are 3 total maps for training, each with 6 randomly distributed caves (as shown in Figures 11). It is assumed that the experiment takes place during the day with an average surface temperature of 22°C (Martian average). Each cave has a temperature drop of 5-10°C at the cave location. The effects of the cave temperature on the surroundings follow a gaussian drop-off law expressed as

$$f(x_{pixel}, y_{pixel}) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{(x_{pixel}-x_{cave})^2+(y_{pixel}-y_{cave})^2}{\sigma}\right)}$$

Where σ = 10 denotes the variance (or reach) of the temperature effect, (x_{pixel}, y_{pixel}) are the coordinates of the pixel, and (x_{cave}, y_{cave}) are the coordinates of the cave. The final temperature values of the pixels given the effect from the cave are

$$T_{pixel} = T_{surface} - (T_{surface} - T_{cave}) f(x_{pixel}, y_{pixel})$$

Where T_{pixel} is the temperature of the pixel, $T_{surface}$ is the surface temperature, and T_{cave} is the cave temperature. This effectively scales the gradient surrounding the cave by a gaussian factor related to the absolute distance from the cave. We can see that in the limit as $f(x)$ tends to 0, the pixel temperature tends to the surface temperature, as desired.

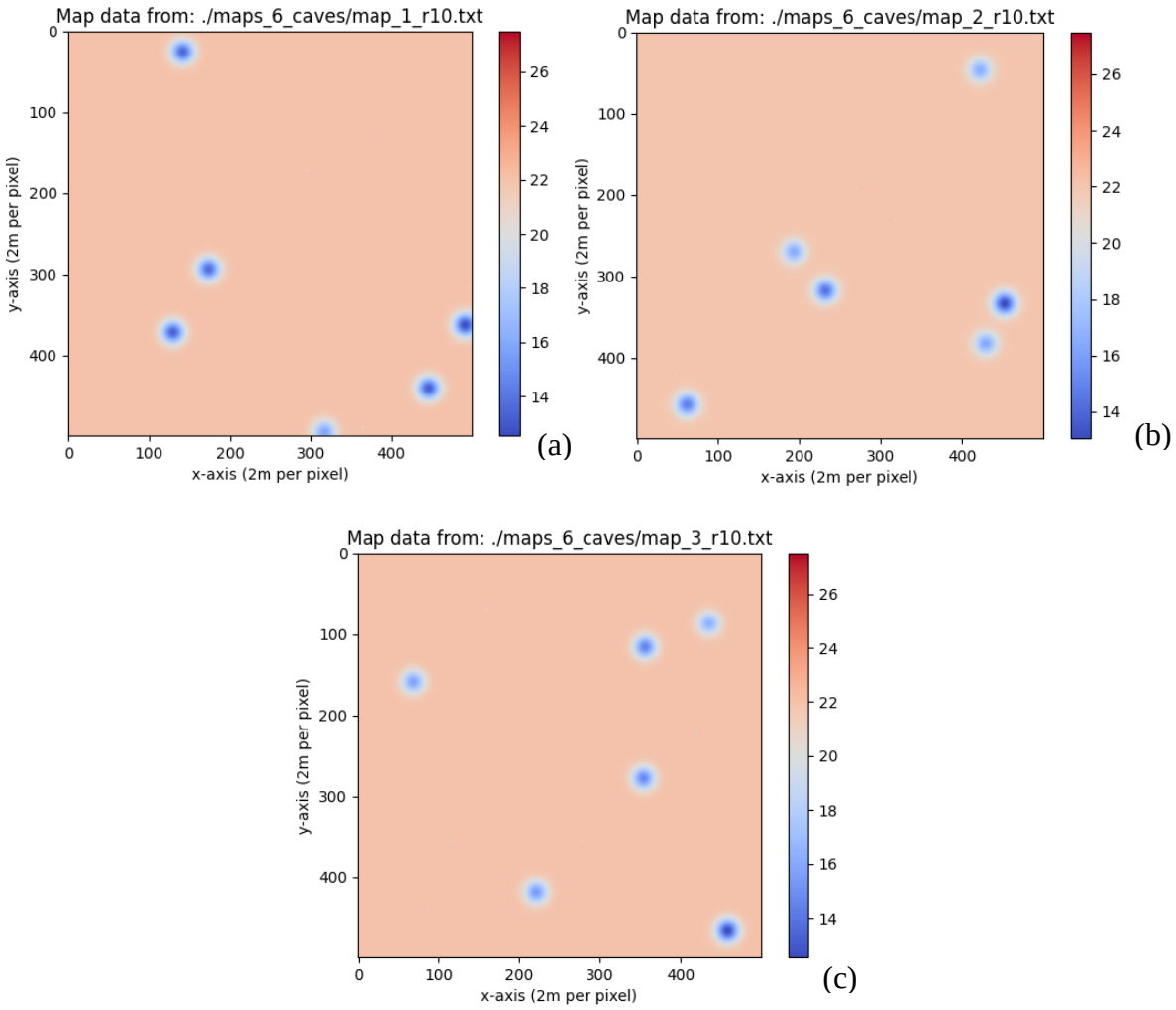


Figure 11: Cave Maps for Training

5.1.2. Markov Brain Agents

Each Markov Brain swarm agent is assumed to represent a UAV agent as proposed by St Onge et al. [68]. Each agent by default has 8 inputs, 4 hidden states, 4 outputs, and 10 gates, as shown in Figure 12. At the beginning of each simulation, the gate types and connections are randomly initialized. Both deterministic and probabilistic (to induce stochasticity) logic gate types are implemented, but special gates such as feedback gates were not implemented due to time constraints.

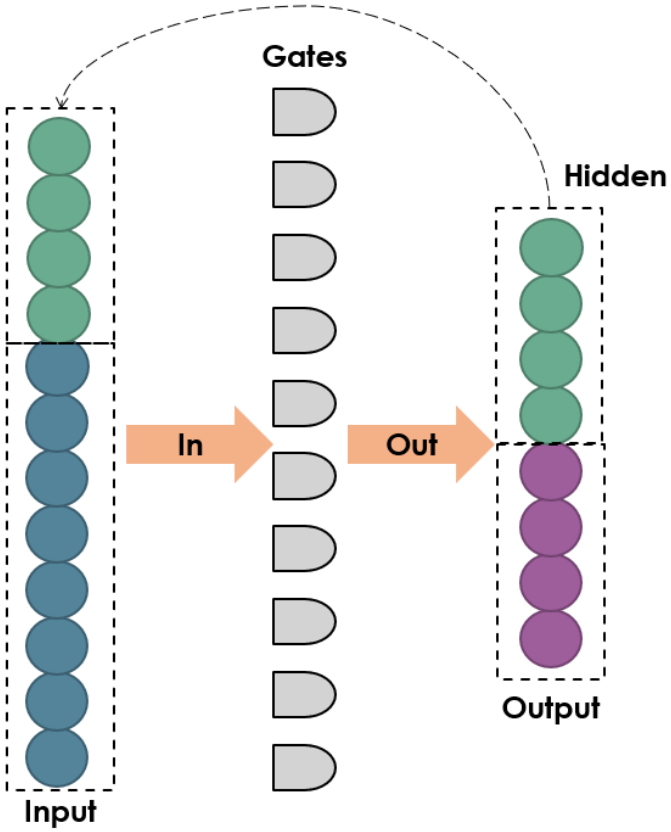


Figure 12: Markov Brain Structure

The 4 outputs represent the agent's ability to move in any combination of 4 directions in a 2D plane. Each agent can move freely into the 8 directly adjacent spaces around it or not move at all. The 8 inputs are divided into two groups: temperature gradient sensing and collision detection. Each of the first 4 inputs detect if another swarm agent is within 2 pixels (4m) of itself in one of the 4 surrounding directions. Each of the last 4 inputs detect if the mean gradient in one of the surrounding 4 directions reaches a threshold of 0.1.

The main challenge with setting up this structure was determining what balance of input, hidden, and output nodes, along with number of gates, was appropriate to represent the agent. Since these numbers remain fixed during evolution, they represent important design choices. Furthermore, the inputs need to provide enough information to the agent to allow it to evolve appropriate behaviour, but also need to be binary. This posed challenges since temperature and distance are not binary values. As such, thresholding methods were leveraged to provide binary inputs. Other methods to do this could include nonlinear activation functions, like in artificial neural networks.

In general, the number of inputs was selected to give the agent knowledge of both collisions and temperature gradients in all directions. The number of outputs were selected to give the agent the ability to move in all directions in the simulation environment. The number of hidden states was determined to match the number of outputs.

The default number of gates were determined experimentally. Since the gates are randomly initialized, too many gates will inevitably result in a high probability of all outputs outputting 1. Too few gates results in all gates outputting 0. The goal was to initialize the Markov Brain with an equal probability of firing either highs or lows. Furthermore, all gates can receive inputs from any of the hidden or input nodes. However, they may only output to 1 of the output or hidden nodes. Since multiple gates may write to the same node, the tiebreaker is determined by a logical OR operation.

In general, any hardware is completely abstracted from the system for simulation purposes. It is assumed that there is some level of onboard processing to manage both inputs from sensors and outputs to actuators.

5.1.3. Evolutionary Algorithm

The Markov Brains in the experiment are evolved by evolutionary algorithm. A general overview of the steps in evolutionary algorithms is provided in Chapter 2 Section 2. The four steps can be broken down within the context of this experiment as follows:

1. **Initialization:** A population of 20 MB agents is randomly initialized. Each of these 20 agents are then run through a simulation to determine the fitness of each agent. Each agent is individually simulated by duplicating the agent 10 times to create the swarm pool for the simulation. The swarm pool is then initialized in a small circle around the center point of the map.
2. **Selection:** After the simulation is conducted, the best performing agent is selected by the average fitness of its swarm. The fitness formulation for **each member** of the swarm pool for **each step** in the simulation can be broken down as

$$\alpha_t = \alpha_{t-1} + \left| x_{grad} \right| + \left| y_{grad} \right| - 30c$$

Where α_t and α_{t-1} are the swarm member's fitness scores in the current and previous timestep respectively, x_{grad} and y_{grad} are the local x and y gradient values respectively, and c is the number of agents within 2 pixels (number of collisions). The fitness is initialized at 0 at the beginning of the simulation. An additional fitness penalty is incurred at the end of simulation for agents that do not move from their initial positions. The fitness of these agents is set to $-\infty$. This effectively prunes agents that preserve fitness by remaining immobile throughout the simulation. Furthermore, the mean distance to the all the other agents is added to the fitness in the final iteration, to promote the spreading of agents across the search space. The final fitness can be summarized as

$$\alpha_n = \begin{cases} -\infty & ,if\ agent\ immobile \\ \alpha_{n-1} + |x_{grad}| + |y_{grad}| - 30c + d_{mean} & ,else \end{cases}$$

Where d_{mean} is the mean distance to the other swarm agents and n is the total number of simulation iterations.

The objectives of these fitness rules are to:

- Promote finding high temperature gradients (caves)
- Avoid collisions
- Promote agent movement (searching behaviour)

It is assumed that an agent is successful at finding a cave if it remains close to the cave for the duration of the experiment. The 2 agents that create swarms with the highest average fitness over the simulation duration are selected to be the parents of the next generation.

3. **Genetic Operations:** The parents undergo crossover and mutation operations to create the next generation of agents. During crossover, a child agent is created with 50% likelihood of receiving individual gates from one parent or the other. The crossover happens at **each individual gate**, with a 50% chance of receiving a gate (and its connections) from either parent. During mutation, each gate of the new child agent has a mutation probability of 1% to be randomly re-initialized. This process is repeated until 20 new agents are produced for the population of the next generation.

4. **Termination:** The evolutionary algorithm terminates after 100 evolution steps (100 generations).

5.2. Simulation Setup

To create the simulation setup, the Modular Agent Based Evolver (MABE) [6], a standardized library/package for creating Markov Brain networks, was considered. However, the library was deemed to be too complex for the purposes of this experiment as it was designed to evolve any kind of brain on any type of world (simulation environment or fitness function). For our purposes, a custom simulation and framework was created in Python. A diagram of the high-level software architecture can be found in Figure 13. The full code can be found at https://github.com/natek1234/MBCaveExplore.

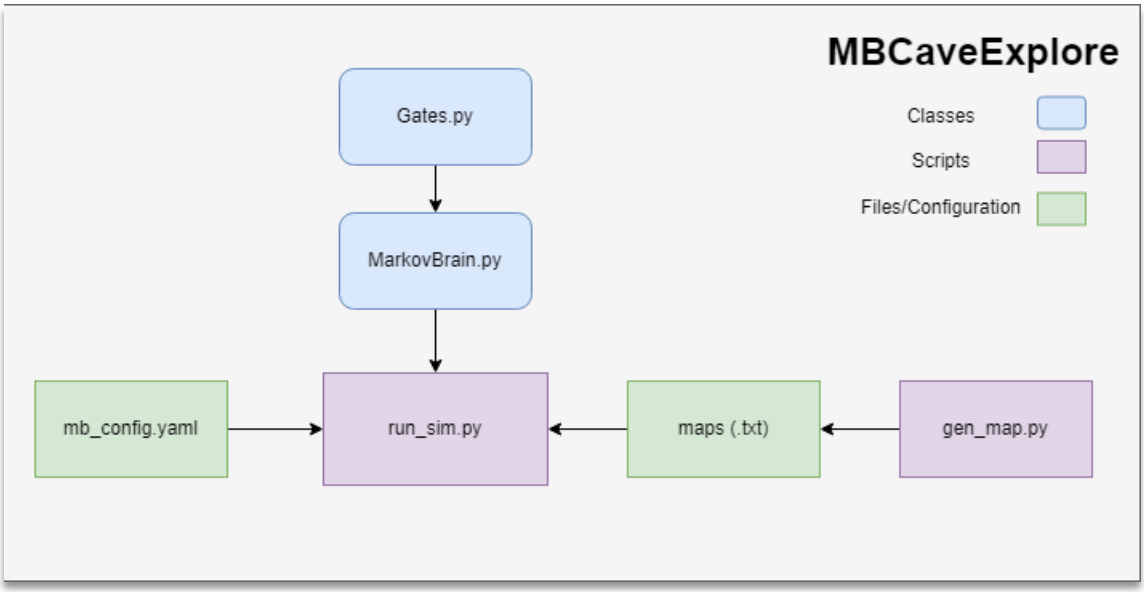


Figure 13: MBCaveExplore Software Architecture

To summarize the functionality, the simulation file (*run_sim.py*) is responsible for running the simulation and evolutionary algorithm. It takes as inputs the configuration file (*mb_config.yaml*), the map files (*maps.txt*), and the Markov Brain class, which also refers to the Gates class which independently manages every gate instance. The maps are generated independently as a pre-processing step (*gen_map.py*). Other tools and functionality are also included in the repository, such as for the creation of fitness curves, for the creation of simulation gifs, and for batch analysis.

The simulation process can be broken down as follows:

1. The configuration file is loaded.
2. The first generation of agents are randomly initialized.
3. For each agent in the population, a swarm pool is generated by duplicating the agent.
4. The swarm pool is initialized at the center of the map in a circle.
5. The simulation iterates over all the simulation timesteps, updating each agent's inputs to produce outputs. The outputs all correspond to movements on the map and the agent locations are updated. The fitness is also updated over the simulation lifetime.
6. Once the simulation has been completed, the average simulation fitness for the swarm is taken as the agent's fitness. Furthermore, the simulation iterates over all three maps to avoid an agent producing behaviour which is only useful on one specific map. The fitness of all agents is averaged over all three maps. The process **returns to step 3** until each map has been simulated.
7. The process moves onto the next candidate agent and **returns to step 3**.
8. Once all agents have been simulated over all the maps, the 2 best agents are selected as parents.
9. The parents undergo crossover and mutation to create the next generation of agents and the process **returns to step 3** for the new generation.
10. Finally, now that all generations have been completed, the line of descent and other statistics are saved to disk. A fitness curve representing the average fitness of the parents is also created and saved.

The entire process is rather expensive since it requires many iterations of the simulation process. To summarize, in every generation all agents must be individually simulated over all three maps. In every generation, the best agent is selected by the average performance over **all three** maps.

Computational Resources

Furthermore, it is worth noting that all simulations were conducted on the Cedar compute cluster from Compute Canada. No GPUs were used as the code has not been optimized

for GPU usage. Every MB was trained using 4 CPUs and one task thread. The batch job script can also be found on the project GitHub.

5.3. Analysis

Finally, once the simulation results have been produced, they require thorough analysis. The major problem with this is the epistemological opacity of the Markov Brain. Like neural networks, much is not understood about the inner workings of Markov Brains or how to represent their effectiveness. Unlike neural networks, the body of research on analysis methods for MBs is still relatively light and unrefined. As such, we must resort to higher-level analysis of the overall system. In this paper, this is done by creating plots which compare the average fitness of the parents of each generation per generation. This measure of performance is compared against differences in various hyperparameters to attempt to understand which parameters have an impact on the brain performance. The selected parameters for analysis are:

- Total evolution steps: 25, 50, **100**, 200, 500
- Mutation probability: **0.01**, 0.03, 0.05, 0.1
- The number of hidden nodes: 2, **4**, 6, 8
- The number of gates: 5, **10**, 15
- The swarm size: 2, 5, **10**, 15
- Total simulation time steps: 250, **500**, 750, 1000

Note: bolded values represent the default configuration values

Manipulating these parameters can help us analyse various properties of our system.

Firstly, investigating the total evolution steps and the mutation probability allows us to investigate whether more time for changes or likelihood of changes to the MB would cause a significant difference in performance. This follows from the biological evolution *punctuated equilibrium theory* that evolution happens in "jumps" resulting from new emergent genes resulting in higher biological success [70].

Secondly, analysing the effect of swarm size can help us understand the effects that the number of swarm agents can have on the success of a swarm. Thirdly, the number of

hidden nodes and gates can help indicate whether increasing model complexity and internal memory has an impact on success. Finally, modifying the total number of simulation time steps can indicate whether agents would have performed better given a different amount of time.

Furthermore, the behaviour of the most successful agents can be analysed to indicate whether swarm behaviour was indeed created. This is important to be able to understand whether we were successful in our attempt to implement certain behaviours. Overall, the analysis objectives can be summarized as follows:

1. Conduct a hyperparameter analysis of the Markov Brain to understand how parameters affects swarm success.
2. Conduct a behavioural analysis of the best brains to understand whether we were successful in creating the desired swarm behaviours.
3. Generally comment on the success of the implementation when compared with the results produced by the honeybee search algorithm [69].

Chapter 6: Experiment Results

The goal of this section is to conduct a thorough analysis of the Markov Brain under various conditions. We will begin by analysing the success of the Markov Brain with different hyperparameters, followed by an analysis of the behaviour and strategies that were observed during simulation. In both cases, the analysis results will be presented, but further discussion will be left for Chapter 7.

6.1. Fitness and Hyperparameters

The first step is to analyse the fitness of the parents per generation against various hyperparameters. To begin, Table 4 presents a hyperparameter matrix for all the candidate hyperparameters. These are compared by the performance (fitness) of the best brain, as well as the overall runtime that was required to train the brain. Runtime considerations are important for both replication of results and future training/implementation. The computational resources for the simulations are explained in Chapter 5 Section 2. It is worth noting that training time may vary depending drastically depending on code optimization and resource availability, but these results should represent general trends.

Note that Table 5 is presented separately (showing the effects of the number of gates on fitness) since the simulations were run locally instead of on the Cedar cluster. As such, no runtime data was collected.

Table 4: Hyperparameter, Fitness, and Training Time Matrix

Hyperparameter	Best Fitness	Training Time (hh:mm:ss)
Total Evolution Steps		
25	71.03	02:02:03
50	154.36	04:38:51
100*	89.66	06:49:49
200	141.00	11:25:40
500	109.50	21:37:02
Mutation Probability		
0.01*	136.08	06:56:21
0.03	114.40	07:34:14
0.05	135.53	06:59:14
0.1	233.96	09:31:19
Number of Hidden Nodes		
2	201.93	05:34:05
4*	93.42	07:28:19
6	54.52	08:47:57
8	103.89	03:16:26
Swarm Size		
2	107.19	02:14:25
5	119.77	04:48:14
10*	174.88	08:58:45
15	78.50	08:38:59
Time Steps		
250	279.44	07:06:54
500*	133.62	06:46:51
750	104.27	08:46:33
1000	205.35	19:35:19

** Default values*

Table 5: Summary of Effect of Gate Size on Fitness

Number of Gates	Fitness
5	279.44
10	133.62
15	104.27

While Table 4 and Table 5 provide a good overview of the trends and performance, it can be more easily visualized through graphs. Figures 14-20 visually depict the effects of varying hyperparameters on the average parent fitness over the evolutionary generations. These figures help us understand which parameters have effects on the overall fitness of the brain and how they might affect the evolutionary behaviour, such as with evolutionary jumps (recall the *theory of punctuated equilibrium*).

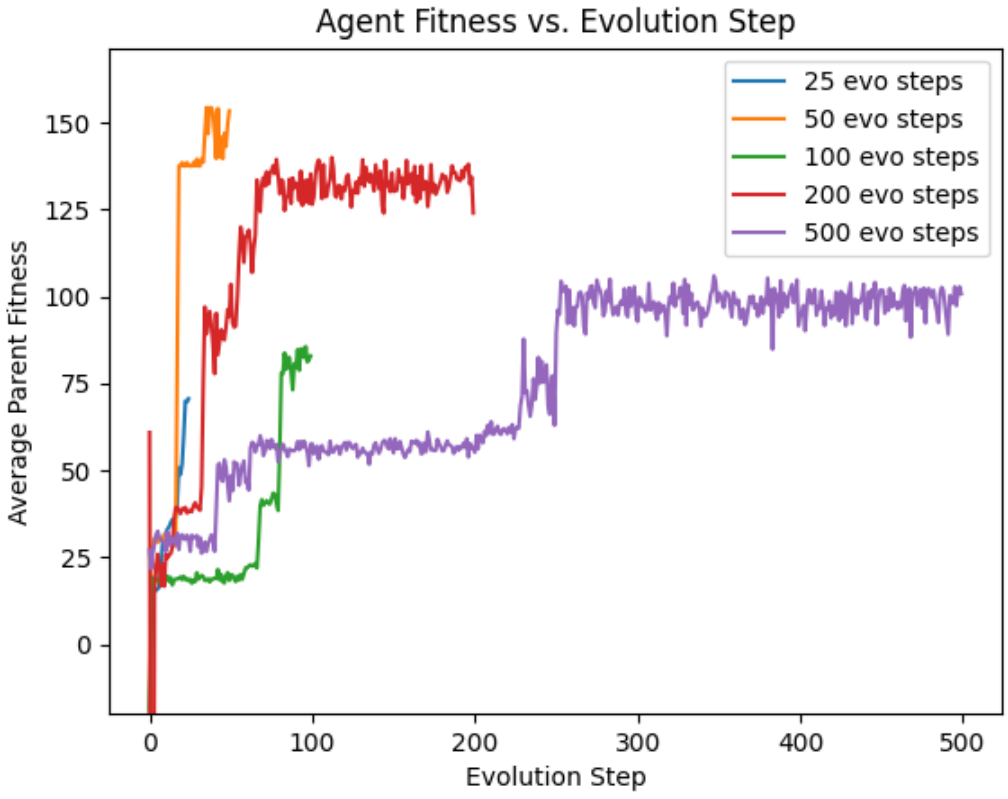


Figure 14: Effects of Total Evolution Steps on Fitness

Figure 14 shows that there is no general correlation between the total number of evolution steps and the absolute (final magnitude) fitness of the parents. However, there

is a clear general trend that shows that as the generations increase for each individual simulation, so does the fitness. Furthermore, the randomness of the evolutionary process can help explain why simply increasing the number of generations in a new simulation might not result in a better final agent. Since initialization and searching (crossover and mutation) of the solution space is random, there is no guarantee that the final performance will be better. Despite this, our results confirm that as the generations increase (for each individual simulation), the fitness tends to increase. It is also worth observing that there are clear "jumps" in fitness which can be attributed to new genes or gene combinations being successful. This might help support the *theory of punctuated equilibrium*, which will be further discussed in Chapter 7.

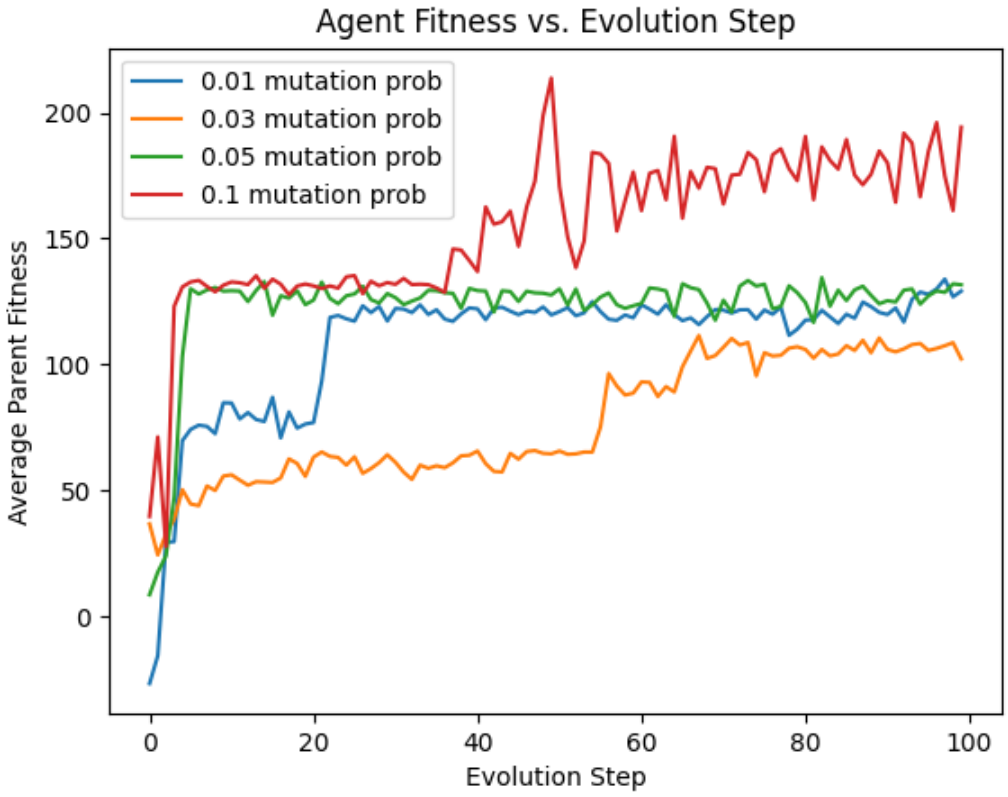


Figure 15: Effects of Mutation Probability on Fitness

From Figure 15, we can see that the simulations for a mutation probability of 0.03, 0.05, and 0.1 depict a clear increasing trend in fitness as the mutation probability is increased. The fact that this is not observed for the mutation probability of 0.01 can be attributed to the randomness of the evolutionary system creating an outlier. This is an interesting result

as it indicates that the evolutionary process might have been getting stuck in local minima within the search space and that increasing the mutation probability helped create gene sequences that could break out of these search space minima. Higher mutation probability can be said to lead to higher behaviour-breaking agents and may help avoid stagnation[1] within a population.

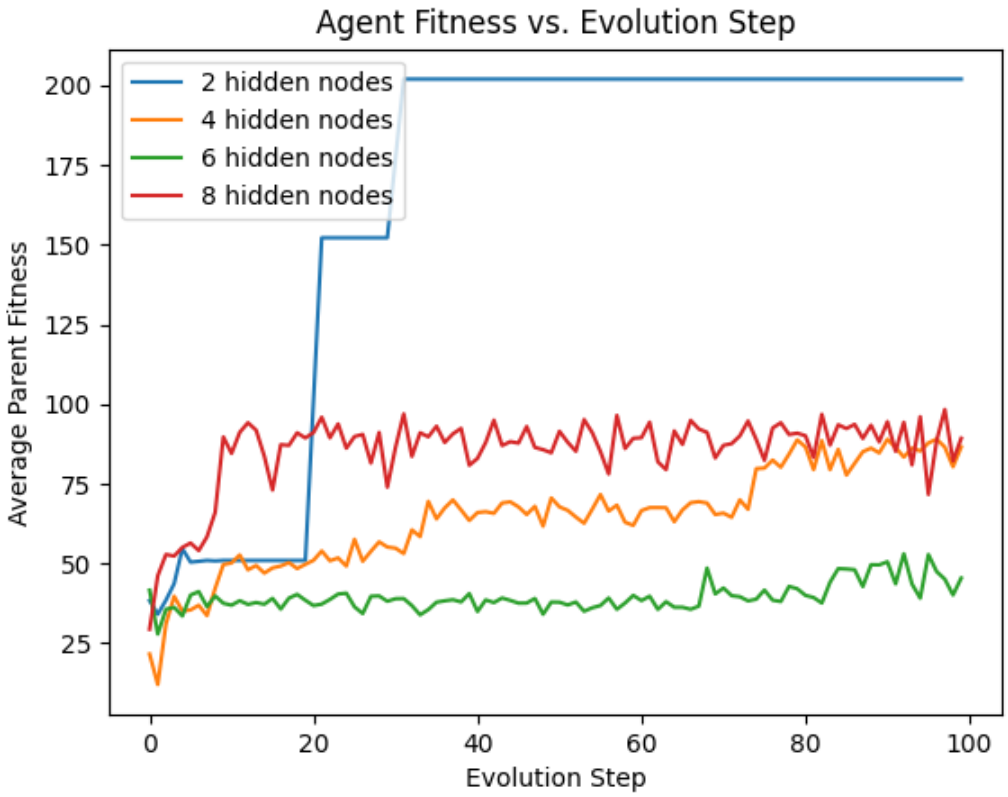


Figure 16: Effects of the Number of Hidden Nodes on Fitness

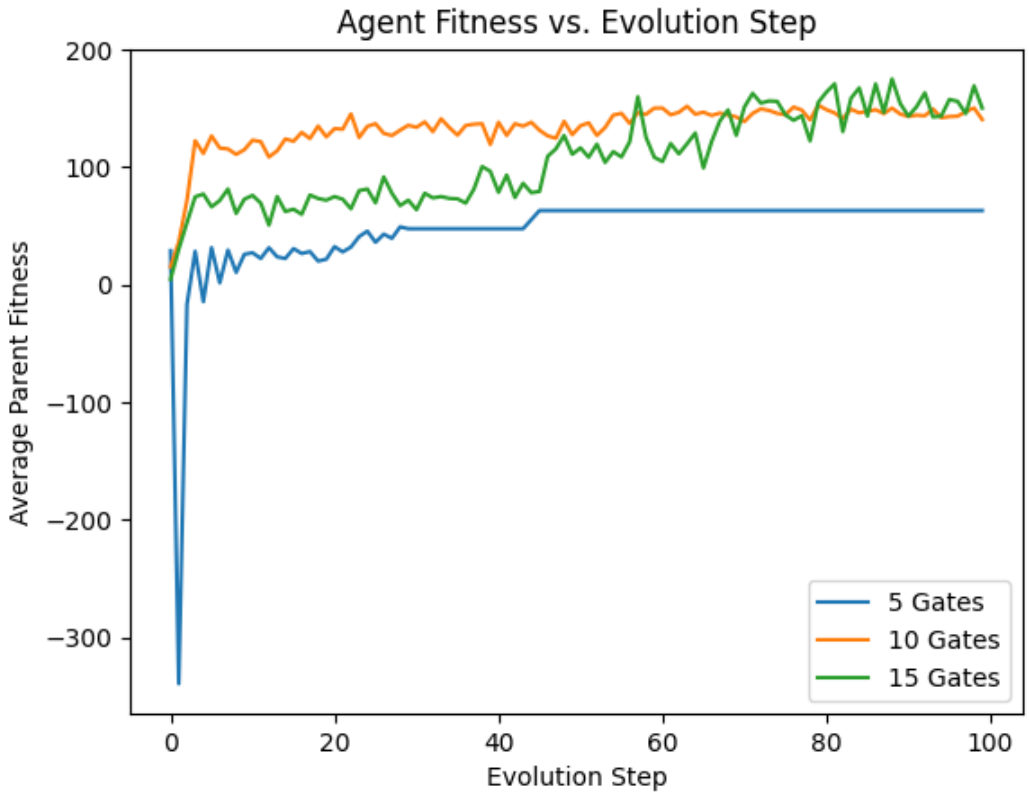


Figure 17: Effects of the Number of Gates on Fitness

Figure 16 and Figure 17 show that in this application, there is no clear trend to suggest that increasing or decreasing the model complexity using the number of hidden nodes or the number of gates has a significant impact on fitness. This might indicate that the success of the model instead depends more on the overall structure of the entire MB, including inputs and outputs, and how these are formulated (the "rules" of the simulation).

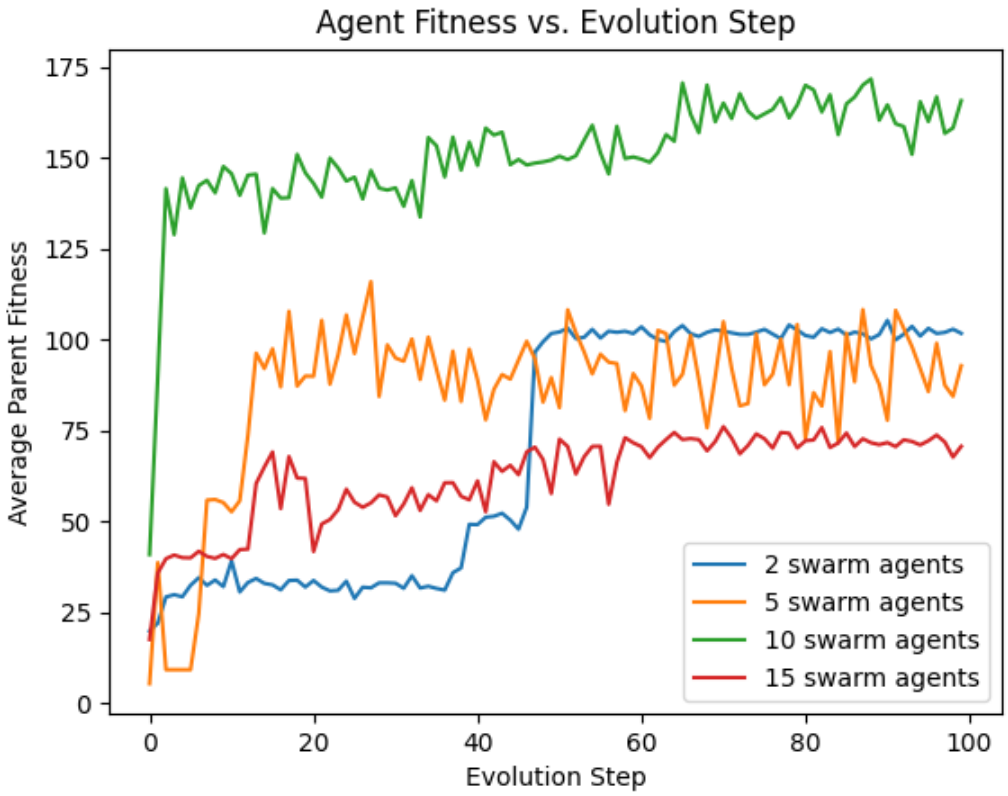


Figure 18: Effects of the Number of Swarm Agents on Fitness

Figure 18 shows that no clear trend is observed between the swarm size and the agent's fitness in this application. This could also be due to the fact that only swarms of 2-15 agents were considered since these were deemed to be reasonable numbers to explore a 1km^2 area. This can potentially be explained by our fitness formulation:

$$\alpha_t = \alpha_{t-1} + |x_{grad}| + |y_{grad}| - 30c$$

We can observe that as the number of agents increase, although more agents may be likely to locate caves (increasing fitness), more agents are also likely to collide (decreasing fitness). This may explain the behaviour observed in Figure 18.

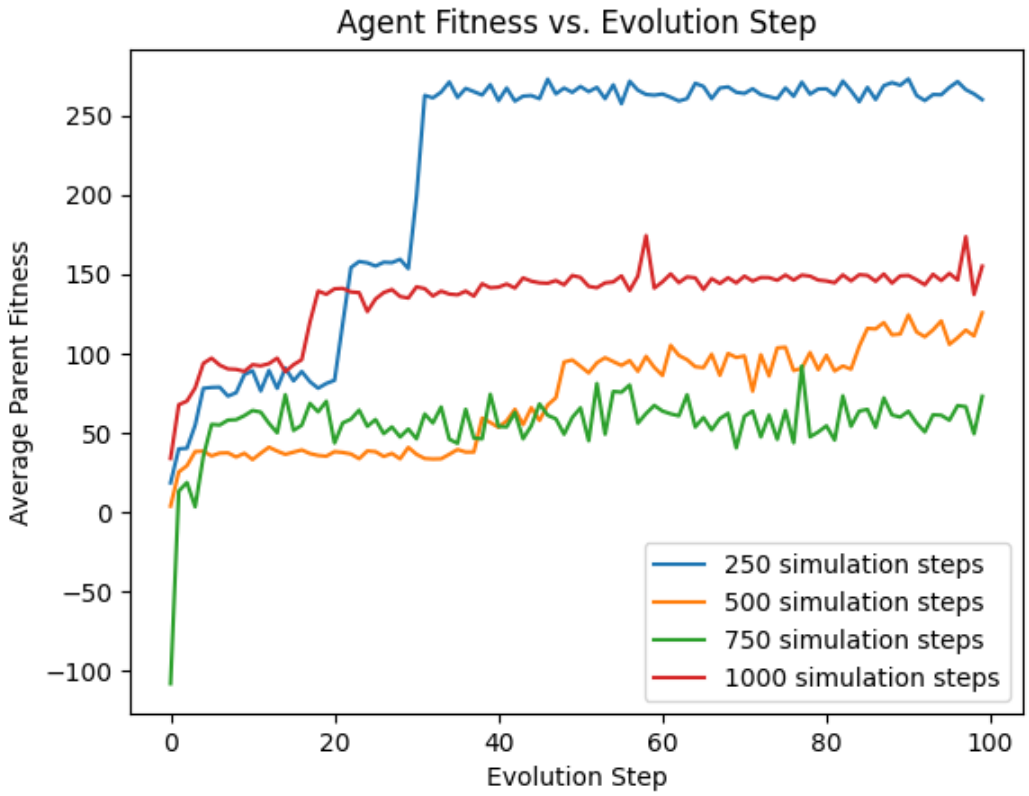


Figure 19: Effects of the Total Number of Simulation Steps on Fitness

Figure 19 shows that there is no correlation between the fitness of the agent and the total number of simulation timesteps. This indicates that agents were able to locate caves even given only 250 timesteps, and that increasing the number of time steps did not result in lower fitness through events such as collisions. However, we can theorize that given an unreasonably low number of timesteps (ex. 10) where an agent would not be capable to reach any of the caves, fitness would suffer.

Another source of variance that was observed in the data was the random nature of the MB initialization and the evolutionary algorithm. This means that although some general trends might be observed, there is no way to nicely characterize these trends (i.e., fit the trends). This also leads to many outliers to trends since some agents might randomly perform better or worse in the exact same configuration due to the randomness of the system. This is depicted in Figure 20 which plots the exact same (default) configuration 6 times, with varying results.

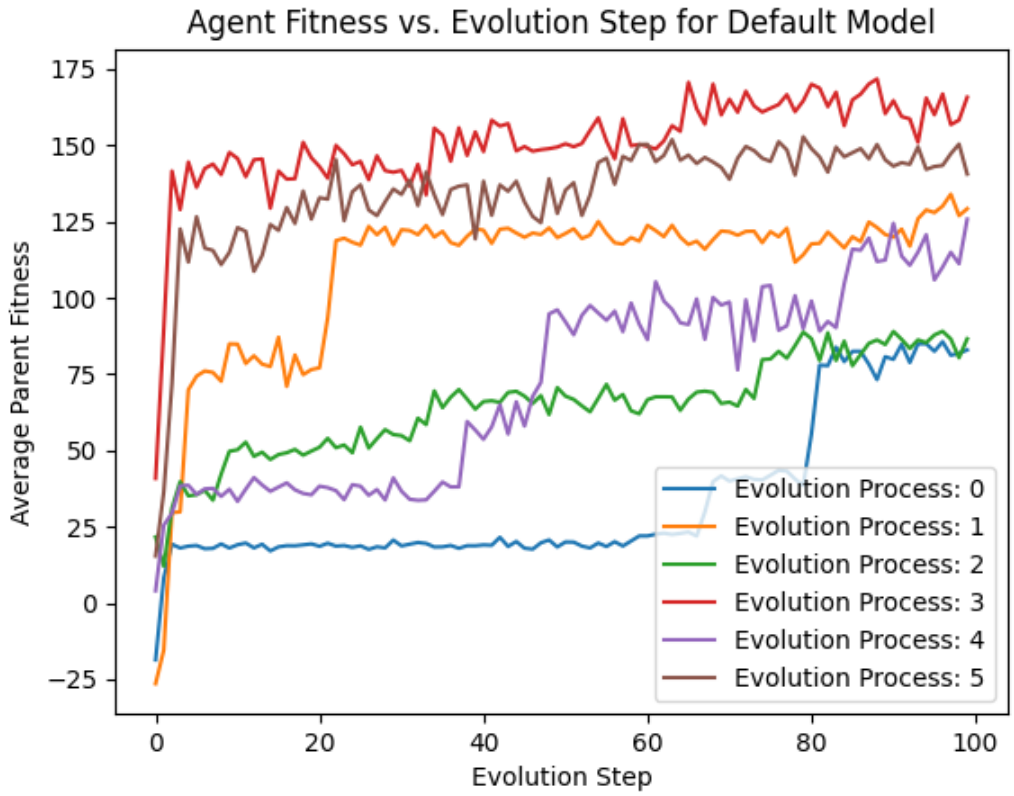


Figure 20: Multiple Iterations of Default Model

As we can see from Figure 20, the random nature of the evolutionary process produces varying results in terms of fitness without changing any parameters at all. The results in terms of fitness vary from the final brains having a fitness of 75-175. In fact, this can be quantified by taking the mean of the variance for each column in the above fitness data. This gives a mean variance $\sigma^2 = 1849.18$ and a standard deviation $\sigma = 43$. This is a very large spread of data due to randomness in the evolutionary process, despite not changing any parameters. However, some general trends do remain. For example, we can observe that the fitness does increase in all cases as the generations increase.

In summary, for our application:

- There is no correlation between the total number of evolution steps and the absolute (final magnitude) fitness of the parents.

- There is a positive correlation between the fitness and the number of generations for each individual simulation. In other words, within the context of a single evolutionary process, as the generations increase, the fitness increases.
- Increases in fitness often occur in sharp “jumps”.
- There is a positive correlation between fitness and mutation probability.
- There is no correlation between model complexity and fitness.
- There is no correlation between swarm size and fitness.
- There is no correlation between the total number of simulation timesteps and fitness.
- The randomness of the evolutionary process can yield varying results despite the exact same model and simulation configuration.
- The randomness of the evolutionary process creates outliers from observed trends.

Please note that all graphs were presented in terms of fitness over generations since this best expresses the behaviour of the fitness throughout the entire evolutionary process.

6.2. Simulated Behaviour

The model can also be analysed from the perspective of the behaviours that were created. For this section, we will focus on swarm strategies and behaviours that were observed in different configurations of the experiment.

One very common strategy that was manifested was that all agents would succeed in discovering caves they crossed paths with, but would only move in one particular direction. This is shown in Figure 21. The hyperparameters in this case were in all the default configurations except for the mutation probability being set to 0.03.

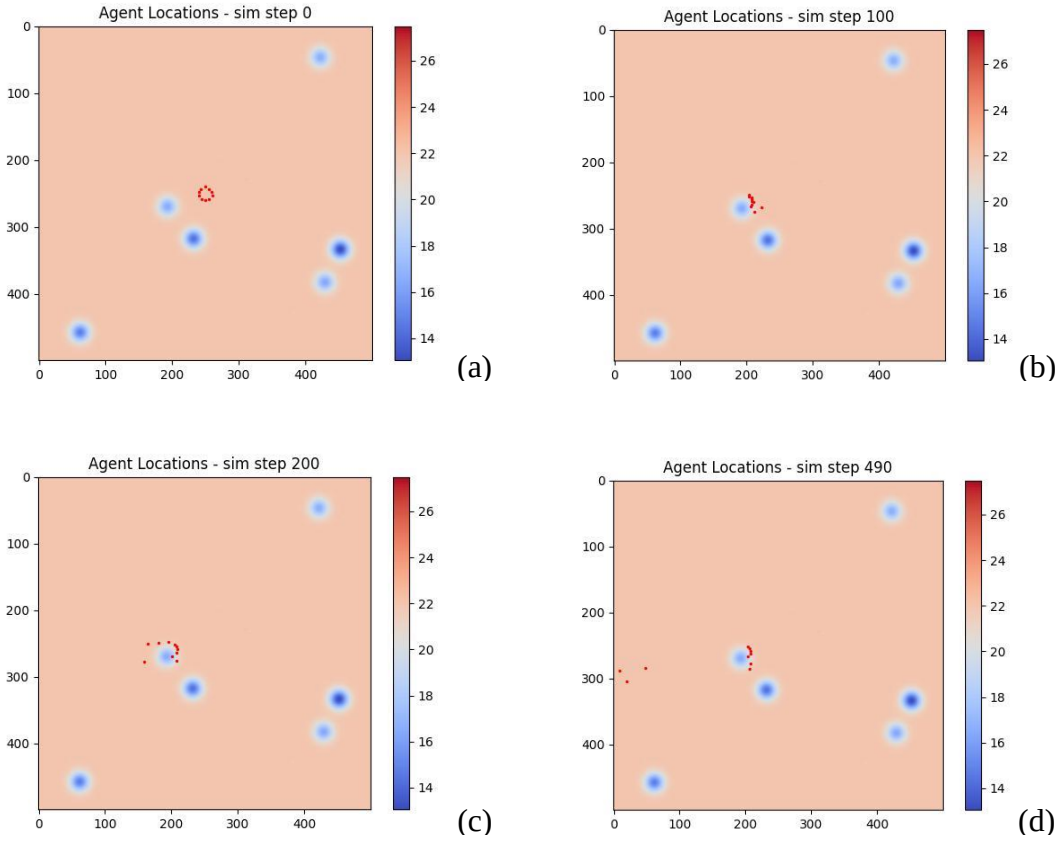


Figure 21: Simulation Results for Mutation Probability 0.03

In Figure 21a we see the starting frame of the simulation. The agents then begin moving to the left in search of caves and Figure 21b shows the frame where agents discover the cave. Some agents move on to keep searching for caves to the left (Figure 21c) while multiple agents remain at the cave location (they "discover" the cave). Finally, Figure 21d shows that the agents that moved on from the cave are unsuccessful at finding more caves.

Another common category of behaviour that was developed was conservative fitness preserving behaviour. Most commonly, agents would maintain their starting shape and distance and only send off a few agents in search of caves (shown in Figure 22). This was interesting since it was likely in response to the collision fitness penalty, showing a direct influence of the fitness formulation on the swarm behaviour.

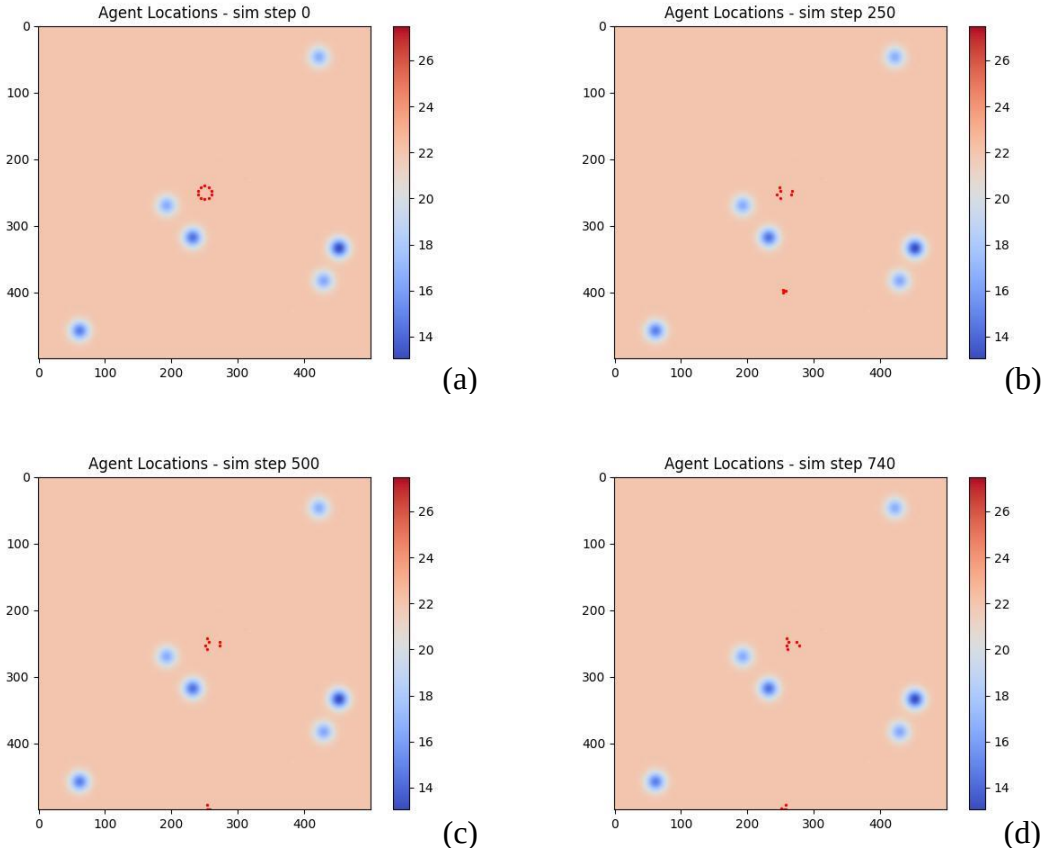


Figure 22: Simulation Results for 750 Simulation Time Steps

Finally, the last but rarest category of behaviour that was observed was forms of random searching with cave detection. This type of behaviour rarely developed, and when it did, the random searching was limited to the vertical direction, as shown in Figure 23. This

indicates that the fitness or model formulation may need to be revisited to better promote true random searching with cave detection.

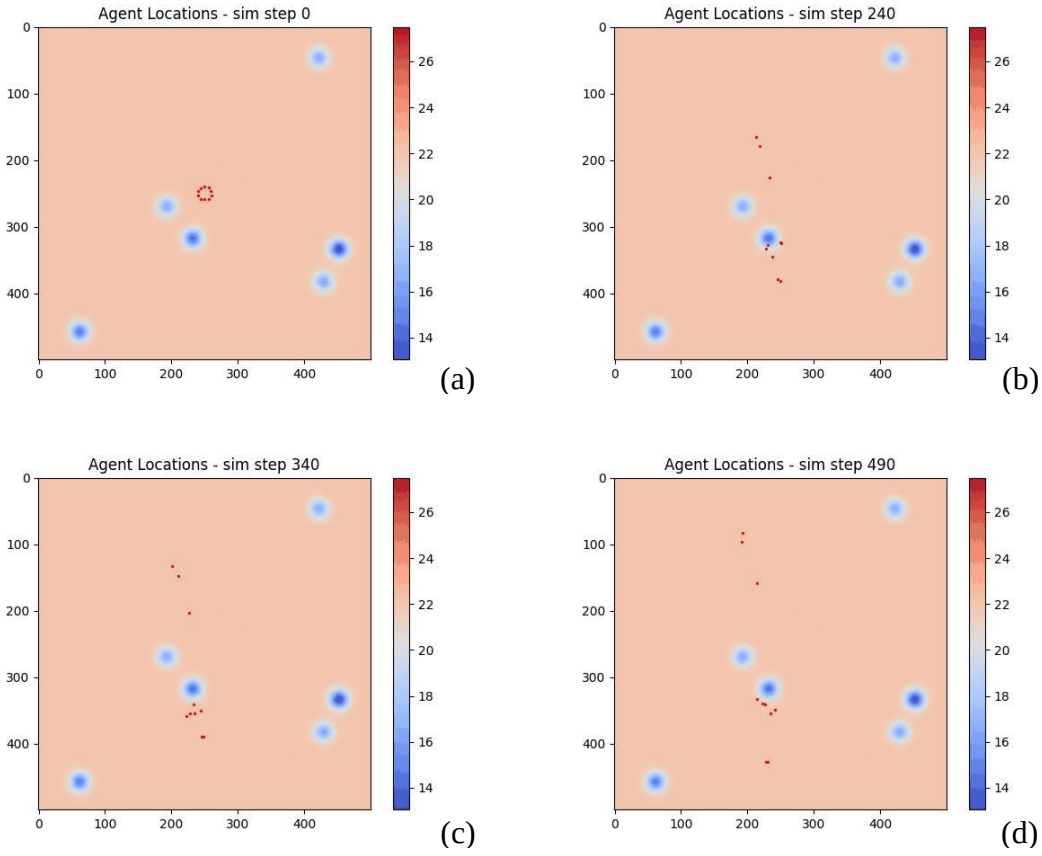


Figure 23: Simulation Results for 15 Gates

In Figure 23b we see that the agents begin to search in both the up and down directions, with a few crossing the cave directly downwards. In Figure 23c we see that all the agents pass the cave, with some remaining close and some going off to search for more caves. In Figure 23d, we see that some agents come back towards the cave after having explored downwards and then remain there until the end of the simulation. This is intriguing behavior since it is a clear demonstration of random searching of caves, albeit only vertically, with some degree of cave detection.

In the interest of conciseness only a few examples were presented. A full set of gifs for every hyperparameter case can be found on the GitHub. We can summarize the types of behaviours that were observed as follows:

1. Cave detection behaviour, but with all agents moving in only one direction (common).
2. Fitness preservation behaviour, such as agents remaining more or less in their starting formation (common).
3. Cave detection behaviour with a degree of random walking (rare).
4. No discernible behaviour or strategy (common). This is essentially a catch-all case for the agents that failed at cave detection and simply moved randomly.

Chapter 7: Discussion

The goal of this chapter will be to investigate and discuss the implications of the results presented in Chapter 6 with regards to this thesis and the research field. The main objectives of the research were to attempt to create a swarm controller using a Markov Brain and to analyse the performance of that brain to further research in the field. As such, our main goals for analysis (from Chapter 5 Section 3) were to investigate the effects of various hyperparameters on the performance of the brain (as measured by fitness), to observe the behaviour of the brains through simulation, and to generally comment on the success of our algorithm when compared to the honeybee search agents created by Kisdi et al. [69]. Throughout our discussion, we will investigate the implications of this work on the field of evolutionary algorithms, swarm robotics, and robotics for space exploration.

7.1. Discussion of Analysis

7.1.1. Discussion of Hyperparameter Analysis

Firstly, we can start with the effects of varying hyperparameters during the MB evolution process. We observed that there was a positive correlation between fitness and the number of generations, within the context of a single evolution process (training a new model for more evolution steps is not guaranteed to achieve higher fitness). This is what we would expect out of an evolutionary algorithm since the overall fitness of a population should grow over time as more successful genes get passed down to the ensuing generations. Eventually, this growth in fitness might plateau, however this was not observed.

We also observed that there was a positive correlation between mutation probability and fitness. This was interesting because one might think that increasing the mutation probability might lead to instability in the fitness as genes are randomly mutated more often. However, this implies that the solution space was heavily constrained and prone to local minima to begin with and that agents benefitted from the additional ability to mutate out of these local minima.

Furthermore, no correlation was observed between model complexity (number of hidden nodes and gates), swarm size, or the number of simulation timesteps with fitness. This

might have been expected for the simulation timesteps and the swarm size since they are parameters more directly related to the simulation than the evolutionary behaviour. However, the lack of correlation was unexpected for the model complexity since one might expect a more complex model might be able to generate more complex behaviour. What this tells us is that the reality is more complicated. Instead, the structure of the entire Markov Brain, including inputs and outputs, is more important than the internal model complexity. This can be considered analogous to neural networks in some ways since simply increasing the number of hidden nodes in a neural network may not increase performance. Instead, it is important to carefully consider how the network is structured to achieve the desired behaviour. Similarly, Markov Brains should be considered holistically during design to ensure that the desired performance and behaviour is achieved.

Another major observation about the evolution process is that its random nature makes it unpredictable. Even when MBs are evolved using the same configuration, it is virtually impossible to guarantee what exact behaviour will result. This is intriguing since it means that the evolutionary algorithm explores a very large solution space and is less likely to get stuck in local solution space minima. The random nature of the process can also produce interesting and useful unanticipated behaviour.

However, the unpredictable nature of the evolution of MBs means that it might not be ideal for engineering design implementation. Engineering design often requires adherence to a very specific set of requirements and objectives. This is even more the case when thinking about engineering for space exploration. While we can attempt to encode these requirements and objectives into the fitness function, one major lesson learned during this research is that the formulation of the fitness function is a difficult task. Seemingly logical constraints can lead to unanticipated behaviour, as is the random nature of evolution. Due to these reasons, it is difficult to imagine that evolutionary design of Markov Brains would be accepted into the engineering design space. To clear this roadblock, more research would need to be done to break through the epistemological opacity of MBs and create a representation and training framework, similar to what was done for neural networks.

7.1.2. Discussion of Cave Detection Swarm Behaviour and Comparisons to the Honeybee Search Algorithm

In terms of the cave detection task, as discussed in Chapter 5 Section 1.3, success in cave detection was determined by whether an agent remained close to the cave throughout the simulation. It is then assumed that researchers or astronauts would have some global knowledge of where the individual swarm agents are located. They would then investigate the areas of interest themselves to confirm that there are indeed caves. In essence, the goal is to achieve aggregation behaviour around potential cave locations. This differs from the honeybee search agents from Kisdi et al. [69] since their agents returned to some central lander to inform on potential cave locations, simulating hive-like behaviour. Another major difference is that their experiment was largely task-oriented, with success metrics like time to detect the first cave, average distance travelled, and the number of caves found. In contrast, this thesis is largely exploratory, focusing less on task-specific success and more on understanding and designing the models that are being implemented within the task-space. As such, the performance metrics are not comparable one-to-one, but some general comparisons can still be drawn.

The main fitness objectives for our Markov Brain agents can be summarized as follows:

1. Follow high temperature gradients towards caves.
2. Avoid collisions at all costs.
3. Promote searching behaviour.

There were 3 main swarm strategies that were observed to have developed during the experiment:

1. Agents collectively move in one direction only while avoiding collisions and stopping at cave locations.
2. Agents display fitness-preserving behaviour such as maintaining the swarm shape to avoid collisions.

3. Agents perform random walk while avoiding collision, but mostly along one axis, while stopping at caves.

Not all the evolved agents exhibited these behaviours. There were many cases where evolved behaviours did not fall under any of these categories and did not accomplish the task.

Behaviour 3 was the closest to the desired behaviour, which would have been a fully random walk in all directions with cave detection and collision avoidance. However, it was the rarest of the observed behaviours. From the perspective of cave detection, we could say these brains were successful at finding caves. However, it is worth noting that more work could have been done to continue with the models that exhibited this category of behaviour to see whether they could develop even better behaviour. The agents that created this behaviour could be used as an evolutionary scaffold in future work to determine if further evolution could produce the desired search behaviour.

In comparison, the agents from Kisdi et al. [69] performed a random walk inspired by bees and consistently detected multiple caves in a relatively short amount of time. In this way, it could be said that the honeybee search agents were more consistent at succeeding at the cave detection task. However, their algorithm has no inherent concept of collision avoidance. In this way, our algorithm could be considered better at collision avoidance.

7.2. Evolutionary Algorithms and Markov Brains

As discussed in the background in Chapter 2, evolutionary algorithms and MBs are biologically inspired algorithms. As such, it is possible to draw some interesting analogies to biology when dissecting the results of this experiment. One important observation that was noted in Chapter 6 Section 1 was that it appears as though the evolution of all the brains observed occurs in sharp “jumps” in fitness. This is an interesting observation since the *punctuated equilibrium theory* states that evolution may happen in “jumps” resulting from new emergent genes resulting in higher biological success [70]. That means that our observation may not only support the theory, but also

help validate evolutionary algorithms and MBs as a proxy for real evolutionary processes.

This indicates that biological analogies for evolution may be a good way to interpret the evolution of a Markov Brain and vice versa. This is important since it means that evolutionary algorithms and Markov Brains could be effectively used by biological evolution researchers to further their understanding of the biological evolution processes. In fact, this is the original inspiration for Markov Brains, particularly for the creators of MABE [6] out of Michigan State University. The focus of their work is on the analysis of artificial evolution systems within the context of biological evolution.

Similarly, researchers in the artificial intelligence field may be able to draw on lessons learned from biological evolution to further their design and understand of Markov Brain evolution. This has been commonly done in similar domains under evolutionary algorithms, such as with Genetic Algorithms, but not as commonly for Markov Brains. In summary, it is worth investigating a shift in mindset within the current Markov Brain research community from how MBs can inform on biological processes, to how biological processes might inform the evolution of MBs. This could lead to breakthroughs in the design of Markov Brains and even lead to a framework for design, which the field is currently lacking.

Chapter 8: Conclusion

Overall, our evolution of Markov Brains did yield brains that exhibited promising behaviour for the task of cave detection. No brain achieved the theoretically ideal behaviour combining a fully random search, cave detection, and collision avoidance. Further evolution of the created brains may result in better performance on the task, but this is not guaranteed. As such, it is not possible at this time to say that the created brains fully achieved all the cave detection task objectives perfectly, and further work and evolution would need to be conducted to determine if it is possible. However, it is possible to say that the evolutionary process yielded brains that are capable of simply searching for and detecting caves.

Our hyperparameter analysis of the MB model discovered some interesting trends in the evolutionary process. Generally, as the evolutionary algorithm completes more generations, fitness tends to increase. Furthermore, the fitness tends to increase in "jumps" which provides some support for the *theory of punctuated equilibrium* for evolution. We also observed that increasing the mutation probability led to increases in fitness. This indicated that increasing the mutation probability helped the algorithm more effectively explore the solution space and avoid getting stuck in local minima.

Markov Brains can develop interesting and unanticipated behaviour through evolution, which makes them ideal proxies for biological evolution research. However, the random nature of Markov Brain evolution, as well as the lack of a clearly defined framework for design, may hinder their acceptance in engineering spaces. Engineering, particularly in the space industry, requires the adherence to a strict set of requirements. Although these requirements can be encoded into the fitness function of the evolutionary algorithm, this is a difficult and non-trivial task. As such, Markov Brains at their current stage of research may not be ideal for implementation in the space industry.

Finally, the work conducted in this thesis can be continued in multiple ways. Future work may include taking the promising brains from this thesis as an evolutionary scaffold to continue the evolution on the cave detection task. It may also include using more computational resources to analyse more hyperparameters and run multiple iterations of the same evolutionary configuration. This would hopefully help account for the outliers created by the randomness of the evolutionary process and help uncover more statistical evidence behind trends and correlations. Lastly, a major challenge that was uncovered

during this thesis was the lack of framework to analyse and design Markov Brains. This is largely due to how relatively new and unexplored the research field is. Creating a standardized mathematical framework for design and analysis would greatly bolster the usability of Markov Brains.

References

[1] "Lunar Living: NASA's Artemis Base Camp Concept – Artemis", Blogs.nasa.gov, 2022. [Online]. Available: https://blogs.nasa.gov/artemis/2020/10/28/lunar-living-nasas-artemis-base-camp-concept/.

[2] "NASA shows off early plans to send astronauts to Mars for 30 days", Space.com, 2022. [Online]. Available: https://www.space.com/nasa-plans-astronauts-mars-mission-30-days.

[3] M. Brambilla, E. Ferrante, M. Birattari and M. Dorigo, "Swarm robotics: a review from the swarm engineering perspective", Swarm Intelligence, vol. 7, no. 1, pp. 1-41, 2013. Available: 10.1007/s11721-012-0075-2

[4] Hintze, A., "Markov Brains: A Technical Introduction", arXiv e-prints, 2017.

[5] A. Hintze, J. Schossau and C. Bohm, "The Evolutionary Buffet Method", Genetic and Evolutionary Computation, pp. 17-36, 2019. Available: 10.1007/978-3-030-04735-1_2

[6] C. Bohm, N. G. and A. Hintze, "MABE (Modular Agent Based Evolver): A framework for digital evolution research", Proceedings of the 14th European Conference on Artificial Life ECAL 2017, 2017. Available: 10.7551/ecal_a_016

[7] G. Spezzano, "Editorial: Special issue 'Swarm robotics,'" Applied Sciences, vol. 9, no. 7, p. 1474, 2019.

[8] M. Brambilla, E. Ferrante, M. Birattari, and M. Dorigo, "Swarm robotics: A review from the Swarm Engineering Perspective," Swarm Intelligence, vol. 7, no. 1, pp. 1–41, 2013. Available: 10.1007/s11721-012-0075-2

[9] J. C. Barca and Y. A. Sekercioglu, "Swarm Robotics reviewed," Robotica, vol. 31, no. 3, pp. 345–359, 2012. Available: 10.1017/S026357471200032X

[10] R. Arnold, K. Carey, B. Abruzzo, and C. Korpela, "What is a robot swarm: A definition for swarming robotics," 2019 IEEE 10th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), 2019.

[11] M. Dorigo, G. Theraulaz, and V. Trianni, "Swarm robotics: Past, present, and future [point of view]," Proceedings of the IEEE, vol. 109, no. 7, pp. 1152–1165, 2021.

[12] D. E. Gbenga and E. I. Ramlan, "Understanding the limitations of particle swarm algorithm for dynamic optimization tasks," ACM Computing Surveys, vol. 49, no. 1, pp. 1–25, 2016.

[13] L. Dou, M. Li, Y. Li, Q.-Y. Zhao, J. Li, and Z. Wang, “A novel artificial bee colony optimization algorithm for Global Path planning of multi-robot systems,” 2014 IEEE International Conference on Robotics and Biomimetics (ROBIO 2014), 2014.
[14] T. Wang and L. Yang, “Beetle swarm optimization algorithm: Theory and application,” Filomat, vol. 34, no. 15, pp. 5121–5137, 2020.
[15] M. J. Krieger, J.-B. Billeter, and L. Keller, “Ant-like task allocation and recruitment in cooperative robots,” Nature, vol. 406, no. 6799, pp. 992–995, 2000.
[16] A. J. Ijspeert, A. Martinoli, A. Billard, and L. M. Gambardella, “Collaboration through the exploitation of local interactions in autonomous collective robotics: The stick pulling experiment,” Autonomous Robots, vol. 11, no. 2, pp. 149–171, 2001.
[17] W. Agassounon, A. Martinoli, and K. Easton, “Macroscopic modeling of aggregation experiments using embodied agents in teams of constant and time-varying sizes,” Autonomous Robots, vol. 17, no. 2/3, pp. 163–192, 2004.
[18] M. Wilson, C. Melhuish, A. B. Sendova-Franks, and S. Scholes, “Algorithms for building annular structures with minimalist robots inspired by brood sorting in ant colonies,” Autonomous Robots, vol. 17, no. 2/3, pp. 115–136, 2004.
[19] V. Trianni, R. Groß, T. H. Labella, E. Şahin, and M. Dorigo, “Evolving aggregation behaviors in a swarm of robots,” Advances in Artificial Life, pp. 865–874, 2003.
[20] O. Soysal and E. Sahin, “Probabilistic aggregation strategies in Swarm Robotic Systems,” Proceedings 2005 IEEE Swarm Intelligence Symposium, 2005. SIS 2005., 2005.
[21] W. M. Spears, D. F. Spears, J. C. Hamann, and R. Heil, “Distributed, physics-based control of swarms of vehicles,” Autonomous Robots, vol. 17, no. 2/3, pp. 137–162, 2004.
[22] B. Shucker and J. K. Bennett, “Scalable control of Distributed Robotic macrosensors,” Distributed Autonomous Robotic Systems 6, pp. 379–388, 2007.
[23] S. Nouyan, A. Campo, and M. Dorigo, “Path formation in a robot swarm,” Swarm Intelligence, vol. 2, no. 1, pp. 1–23, 2007.
[24] P. M. Maxim, W. M. Spears, and D. F. Spears, “Robotic Chain Formations,” IFAC Proceedings Volumes, vol. 42, no. 22, pp. 19–24, 2009.
[25] V. Sperati, V. Trianni, and S. Nolfi, “Self-organised path formation in a swarm of Robots,” Swarm Intelligence, vol. 5, no. 2, pp. 97–119, 2011.

[26] R. Gross and M. Dorigo, “Self-assembly at the macroscopic scale,” Proceedings of the IEEE, vol. 96, no. 9, pp. 1490–1508, 2008.
[27] R. L. Stewart and R. A. Russell, “A distributed feedback mechanism to regulate wall construction by a robotic swarm,” Adaptive Behavior, vol. 14, no. 1, pp. 21–51, 2006.
[28] A. Howard, M. J. Matarić, and G. S. Sukhatme, “Mobile Sensor Network deployment using potential fields: A distributed, scalable solution to the area coverage problem,” Distributed Autonomous Robotic Systems 5, pp. 299–308, 2002.
[29] G. A. Di Caro, F. Ducatelle, and L. M. Gambardella, “Wireless Communications for distributed navigation in Robot Swarms,” Lecture Notes in Computer Science, pp. 21–30, 2009.
[30] F. Ducatelle, G. A. Di Caro, C. Pinciroli, F. Mondada, and L. M. Gambardella, “Communication assisted navigation in robotic swarms: Self-organization and Cooperation,” 2011 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2011.
[31] E. Ferrante, A. E. Turgut, C. Huepe, A. Stranieri, C. Pinciroli, and M. Dorigo, “Self-organized flocking with a mobile robot swarm: A novel motion control method,” Adaptive Behavior, vol. 20, no. 6, pp. 460–477, 2012.
[32] G. Baldassarre, S. Nolfi, and D. Parisi, “Evolving Mobile Robots able to display collective behaviors,” Artificial Life, vol. 9, no. 3, pp. 255–267, 2003.
[33] R. Gross and M. Dorigo, “Towards group transport by swarms of Robots,” International Journal of Bio-Inspired Computation, vol. 1, no. 1/2, p. 1, 2009.
[34] A. Campo, S. Nouyan, M. Birattari, R. Groß, and M. Dorigo, “Negotiation of goal direction for Cooperative Transport,” Ant Colony Optimization and Swarm Intelligence, pp. 191–202, 2006.
[35] Á. Gutiérrez, A. Campo, F. Monasterio-Huelin, L. Magdalena, and M. Dorigo, “Collective decision-making based on social odometry,” Neural Computing and Applications, vol. 19, no. 6, pp. 807–823, 2010.
[36] S. Garnier, J. Gautrais, M. Asadpour, C. Jost, and G. Theraulaz, “Self-organized aggregation triggers collective decision making in a group of cockroach-like robots,” Adaptive Behavior, vol. 17, no. 2, pp. 109–133, 2009.
[37] W. Agassounon and A. Martinoli, “Efficiency and robustness of threshold-based distributed allocation algorithms in multi-agent systems,” Proceedings of the first

international joint conference on Autonomous agents and multiagent systems part 3 - AAMAS '02, 2002.
[38] A. N. Sloss and S. Gustafson, “2019 evolutionary algorithms review,” Genetic Programming Theory and Practice XVII, pp. 307–344, 2020.
[39] D. Simon, Evolutionary Optimization Algorithms. John Wiley & Sons, 2013.
[40] S. Luke, Essentials of Metaheuristics. Lulu, 2013.
[41] D. E. Goldberg, Genetic algorithms in search, optimization, and machine learning. Boston: Addison-Wesley, 2012.
[42] R. Poli, W. B. Langdon, N. F. McPhee, and J. R. Koza, A field guide to Genetic Programming. Lieu de publication non identifié: Lulu Press, 2008.
[43] J. Petke, S. O. Haraldsson, M. Harman, W. B. Langdon, D. R. White, and J. R. Woodward, “Genetic improvement of software: A comprehensive survey,” IEEE Transactions on Evolutionary Computation, vol. 22, no. 3, pp. 415–432, 2018.
[44] M. Fenton, J. McDermott, D. Fagan, S. Forstenlechner, E. Hemberg, and M. O'Neill, “PONYGE2,” Proceedings of the Genetic and Evolutionary Computation Conference Companion, 2017.
[45] M. Brameier and W. Banzhaf, Linear Genetic Programming. New York: Springer, 2011.
[46] J. Miller, Cartesian genetic programming. Scholars Portal.
[47] L. Spector, N. F. McPhee, T. Helmuth, M. M. Casale, and J. Oks, “Evolution evolves with autoconstruction,” Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion, 2016.
[48] K. Stanley and J. Clune, “Welcoming the Era of Deep Neuroevolution,” Uber Blog, 2017.
[49] O. Chang and H. Lipson, “Neural network quine,” The 2018 Conference on Artificial Life, 2018.
[50] L. Türkler, T. Akkan, and L. Ö. Akkan, “Usage of evolutionary algorithms in Swarm Robotics and design problems,” Sensors, vol. 22, no. 12, p. 4437, 2022.
[51] L. Sheneman and A. Hintze, “Machine Learned Learning Machines,” arXiv, 2017.
[52] A. Tehrani-Saleh, T. LaBar, and C. Adami, “Evolution leads to a diversity of motion-detection neuronal circuits,” arXiv, 2018.

[53] C. Bohm, S. Albani, C. Ofria, and A. Ackles, “Using the comparative hybrid approach to disentangle the role of substrate choice on the evolution of cognition,” Artificial Life, vol. 28, no. 4, pp. 423–439, 2022.
[54] L. Marstaller, A. Hintze, and C. Adami, “The evolution of representation in simple cognitive networks,” Neural Computation, vol. 25, no. 8, pp. 2079–2107, 2013.
[55] J. A. Edlund, N. Chaumont, A. Hintze, C. Koch, G. Tononi, and C. Adami, “Integrated information increases with fitness in the evolution of Animats,” PLoS Computational Biology, vol. 7, no. 10, 2011.
[56] J. Schossau, C. Adami, and A. Hintze, “Information-theoretic neuro-correlates boost evolution of Cognitive Systems,” Entropy, vol. 18, no. 1, p. 6, 2015.
[57] D. Koller and N. Friedman, Probabilistic graphical models principles and Techniques. Cambridge, Mass: MIT Press, 2012.
[58] R. S. Olson, A. Hintze, F. C. Dyer, D. B. Knoester, and C. Adami, “Predator confusion is sufficient to evolve swarming behaviour,” Journal of The Royal Society Interface, vol. 10, no. 85, p. 20130305, 2013.
[59] P. Kvam, J. Cesario, J. Schossau, H. Eisthen, and A. Hintze, “Computational evolution of decision-making strategies,” arXiv, 2015.
[60] D. Fischer, S. Mostaghim, and L. Albantakis, “How swarm size during evolution impacts the behavior, generalizability, and brain complexity of animats performing a spatial navigation task,” Proceedings of the Genetic and Evolutionary Computation Conference, 2018.
[61] A. Hintze and J. Schossau, “Towards an FPGA accelerator for Markov brains,” The 2022 Conference on Artificial Life, 2022.
[62] A. Tehrani-Saleh, J. D. McAuley, and C. Adami, “Mechanism of duration perception in artificial brains suggests new model of attentional entrainment,” 2019.
[63] L. Albantakis, “A tale of two Animats: What does it take to have goals?,” The Frontiers Collection, pp. 5–15, 2018.
[64] D. Kirkpatrick and A. Hintze, “The evolution of representations in genetic programming trees,” Genetic Programming Theory and Practice XVII, pp. 121–143, 2020.

[65] P. S. Schenker, T. L. Huntsberger, P. Pirjanian, E. T. Baumgartner, and E. Tunstel, “Planetary Rover Developments Supporting Mars Exploration, Sample Return and Future Human-Robotic Colonization,” Autonomous Robots, vol. 14, no. 2/3, pp. 103–126, 2003.
[66] A. Owens and N. Singh, "Perspectives on the future of space exploration," McKinsey Insights, 2017.
[67] A. Petrovsky, I. Kalinov, P. Karpyshev, D. Tsetserukou, A. Ivanov, and A. Golkar, “The two-wheeled robotic swarm concept for Mars exploration,” Acta astronautica, vol. 194, pp. 1–8, 2022.
[68] D. St-Onge, M. Kaufmann, J. Panerati, B. Ramtoula, Y. Cao, E. B. J. Coffey, and G. Beltrame, “Planetary exploration with robot teams: Implementing higher autonomy with swarm intelligence,” IEEE Robotics & Automation Magazine, vol. 27, no. 2, pp. 159–168, 2020.
[69] Á. Kisdi and A. R. L. Tatnall, “Future robotic exploration using Honeybee search strategy: Example search for caves on Mars,” Acta Astronautica, vol. 68, no. 11-12, pp. 1790–1799, 2011.
[70] C. Venditti and M. Pagel, “Speciation and bursts of evolution,” Evolution: Education and Outreach, vol. 1, no. 3, pp. 274–280, 2008.